



南京大學

研究生畢業論文

(申請碩士學位)

論文題目 基于估錯碼的数据纠错机制研究

作者姓名 魏兴慎

学科、专业名称 计算机软件与理论

研究方向 分布式计算与并行处理

指导教师 李文中 副教授 陆桑璐 教授

2013年5月

学 号: **MG1033043**

论文答辩日期: **2013 年 5 月 30 日**

指 导 教 师: **陆 子 明** (签字)

Correcting Erroneous Data Bits Based on Error Estimating Code

by

Xingshen Wei

Directed by

Associate Professor Wenzhong Li, Professor Sanglu Lu

**Department of Computer Science and Technology
Nanjing University**

May 2013

*Submitted in partial fulfilment of the requirements
for the degree of Master in Computer Software and Theory*

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目： 基于估错码的数据纠错机制研究

计算机软件与理论 专业 2010 级硕士生姓名： 魏兴慎

指导教师（姓名、职称）： 李文中副教授 陆桑璐教授

摘 要

纠错技术是保障无线网络可靠通信的重要技术之一。现有的纠错码技术可以在不重传数据的情况下，修复出错的数据位，但由于它具有较高的数据冗余度和计算复杂性，会降低网络的传输效率。近年来，研究人员提出了估错码（Error Estimating Code，简称为EEC）技术，只需要在数据包中附加少量的冗余位，就可以快速估算误码率。但是，估错码不具备纠错能力，当误码发生时，仍然需要重传整个数据包。

本论文探讨了基于EEC的数据纠错能力，研究当利用EEC检测到误码时，能否在不重传整个数据包的情况下，纠正该数据包的出错数据位。我们的研究表明，大部分出错数据包的误码率较小（低于0.06）。对于这种误码率小的数据包，可以利用EEC编码的概率特性，纠正其中大部分的出错数据位（修复率可达80%以上）。

我们基于EEC校验位的校验信息提出了数据纠错的相关策略。首先通过引入过滤算法，将正确的比特排除掉并得到一个可疑比特集合，该集合中包含了大多数的出错比特。然后利用具有多项式复杂度的随机翻转算法进一步测试这些可疑比特，找出最可能出错的比特，从而最小化数据包中出错比特数目。理论分析显示该随机算法在误码率比较低时，能够以高概率修复大多数的错误；基于真实WiFi访问日志的性能评估也显示了该算法的有效性。

其次，通过探究真实的WiFi访问日志发现，数据包中错误数量的分布符合幂律分布，同时数据包中的错误往往是突发错误。结合上述发现，我们将估错码应用到出错数据包修复的问题中，设计出一种新的协议EEC-PPR。该协议使用估错码和分组校验码对数据包进行编码。在对出错数据包进行修复时，根据估算的误码率，选取不同的纠错策略：误码率足够小则选用估错码直接纠错，当误码率较大时，则选择重传某些分组纠错。该协议充分利用了估错码的估算技术和纠错策略，因此避免了大量数据包的重传，使用的分组重传技术避免了因重传整个数据包造成的资源浪费。基于WiFi访问日志，我们对比了不同的出错数据包修复技术，结果显示该协议可以极大地提高数据传输效率。

本文的主要工作及贡献包括以下几个方面：

- 给出基于估错码的纠错问题的形式化描述。通过将估错码的纠错问题形式化为优化问题，在给定条件下，通过优化目标函数，纠正数据包中的错误。
- 提出了一种基于估错码的数据纠错策略。该策略首先通过过滤器算法得到可疑比特集合，其次使用随机翻转算法依照特定概率翻转可疑数据比特，以最小化优化目标函数，纠正出错的数据位。理论分析证明当误码率足够小时，该算法能够在较大概率上纠正绝大部分出错的数据位。
- 提出了一种新的出错数据包修复协议EEC-PPR。基于较低冗余的编码，该协议利用GS-EEC的估错和纠错能力，有效减少数据的重传次数，并利用分组编码有效减少重传的数据量。
- 基于真实WiFi访问日志进行性能评估。利用真实的WiFi访问日志，我们评估了所提出的纠错策略和出错包修复修复协议EEC-PPR 的性能，结果显示策略能有效地降低数据包中的误码率，EEC-PPR能减少重传，降低了系统时延，提升网络性能。

关键词： 估错码; 出错数据包修复; 无线网络

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Correcting Erroneous Data Bits Based on Error Estimating Code

SPECIALIZATION: Computer Software and Theory

POSTGRADUATE: Xingshen Wei

MENTOR: Associate Professor Wenzhong Li, Professor Sanglu Lu

Abstract

Error Correction Code (ECC) is an important technique to guarantee reliable communication in wireless networks. Currently, ECC techniques introduce the benefit of error correction without retransmitting the data packet, but they suffer from high redundancy and communication overhead, which decreases network transmission efficiency. In the recent years, error estimating code (EEC) was proposed to estimate the bit-error-rate (BER) of a packet efficiently with very low data redundancy. However, since EEC does not have error correction ability, the entire packet is retransmitted to correct erroneous bits, when errors occur.

This paper discusses the error correction based on EEC and investigates when EEC detects errors, whether the error bits could be corrected without retransmitting the entire packet. Our research result shows that, bit-error-rate in most of erroneous packets is small(smaller than 0.06). By utilizing the EEC probabilistic characteristics, most of error bits in these packets with small BER can be corrected (the recovery rate can reach more than 80%)

We propose an error correction scheme based on the parity check information provided by the EEC bits. Firstly, the filtering algorithm is introduced to rule out the correct data bits and obtain a set of suspicious bits containing most of the errors. Then a polynomial randomized algorithm called *Rand_flipping* is applied to examine the suspicious bits and flip the most promising erroneous bits aiming to minimize the total numbers of errors in the packet. We present theoretical analysis to the algorithm and prove that *Rand_flipping* can correct most of the erroneous bits with high probability when BER is small enough; performance evaluation based on a real WiFi trace shows the effectiveness of the proposed algorithms.

By exploring the real WiFi trace, we find that the distribution of BER in the packets follows the power law distribution and errors are always burst, i.e. they tend to occur continuously and batchly. Motivated by the above findings, we design a novel protocol called EEC-PPR to recovery partial packet in wireless networks. The protocol first encodes the original packet by both EEC and block check codes. Depending on the estimating value of BER and a predefined threshold, the protocol recovers partial packet with EEC error correction scheme when BER is small enough, or, alternatively block-retransmission scheme when BER is high. The protocol avoids a plurality of packet retransmission, by employing EEC error correction scheme, decreases the wastage caused by the entire packet retransmission by utilizing block based parity check. Comparison of different partial packet recovery based on a real WiFi trace shows the effectiveness of our protocol.

The main contributions of this thesis are as follows:

1. We formulate the error correction problem based on Error Estimation Code. By formalization of using EEC to recovery the errors bits as an optimization problem, error bits in the packet are recovered through optimizing the objective function.
2. We proposed an error correction scheme based on EEC. This scheme introduces the filtering algorithm to obtain a set of suspicious bits. Then, the randomized algorithm called *Rand_flipping* is used to flip the most possible erroneous bits in the suspicious bit set aiming to minimize the total numbers of errors in the packet. Theoretical analysis proves that our scheme can correct most of the erroneous bits with high probability when the bit-error-rate is small enough.
3. We Proposed a partial packet recovery protocol for wireless communications: EEC-PPR. Based on low redundancy code, this protocol effectively reduces the number of packet retransmission, by utilizing the error correction ability of EEC, and effectively decreases the amount of retransmission data by employing block code.
4. We conduct performance evaluation based on a real WiFi trace, which shows the effectiveness of the proposed algorithms.

Keywords: Error Estimating Code; Partial Packet Recovery; Wireless Networks

目 录

目录	v
第一章 绪论	1
1.1 研究背景	1
1.2 问题分析	2
1.3 本文主要的工作	3
1.4 本文的组织结构	6
第二章 相关工作	9
2.1 信道编码及差错控制	9
2.1.1 检错码	9
2.1.2 纠错码	10
2.1.3 差错控制	14
2.2 估错码	17
2.2.1 估错码的设计理念	17
2.2.2 GS-EEC编码解码方案	19
2.2.3 RAK-EEC编码解码方案	21
2.2.4 EToW-Sketch编码解码方案	22
2.2.5 AMPS编码解码方案	24
2.2.6 Generalized-EEC的编码和解码	25
2.3 数据包的错误修复协议	26
2.3.1 不使用物理层信息的错误修复技术	28
2.3.2 使用物理层信息的错误修复技术	30
第三章 基于估错码的数据纠错算法	33
3.1 基于估错码的错误修复	34
3.2 错误修复技术	36
3.3 过滤器算法	37

3.4	随机翻转的错误修复算法	40
3.5	性能评估	42
3.5.1	WiFi访问日志	43
3.5.2	过滤器的有效性评估	44
3.5.3	随机翻转算法的性能	44
3.6	本章小结	45
第四章	EEC-PPR: 基于估错码的数据包修复协议	47
4.1	概述	47
4.2	WiFi信道的错误分布	49
4.2.1	数据包误码率符合幂律分布	49
4.2.2	WiFi信道中的错误类型	51
4.3	数据包修复协议的设计	54
4.3.1	估错与分组	54
4.3.2	对估错算法的更新	58
4.3.3	基于估错码的自动重传请求	61
4.4	性能评估	63
4.5	本章小结	67
第五章	总结与展望	69
附录 A	WiFi访问日志收集的真实场景设置	73
参考文献		75
简历与科研成果		81
致谢		83

表 格

表 4.1 真实WiFi访问日志中出错数据包中的误码率统计表一 50

表 4.2 真实WiFi访问日志中出错数据包中的误码率统计表二 50

插图

图 2.1 三种常见的差错控制方式	13
图 2.2 自动重传请求流程图	14
图 2.3 停止等待ARQ	15
图 2.4 后退N-ARQ	16
图 2.5 GS-EEC的编码算法示意图	19
图 2.6 RAK-EEC 编码方案	22
图 3.1 纠错策略的流程	34
图 3.2 误码率与优化目标函数 $f(C, -1)$ 的关系	36
图 3.3 随机选取的比特的 $\psi(d'_i)$ 函数的值	40
图 3.4 出错比特和正确比特 $\psi(d'_i)$ 函数的值的分布	41
图 3.5 过滤器的性能	43
图 3.6 随机翻转算法的性能	44
图 3.7 随机翻转算法前后访问日志中误码率的累计分布图	45
图 4.1 当前802.11中的自动重传请求	47
图 4.2 基于EEC的出错包修复EEC-PPR	48
图 4.3 数据包中误码率的分布	50
图 4.4 数据包中误码率的log-log分布	50
图 4.5 WiFi误码率的概率分布	51
图 4.6 WiFi误码率的累积分布	51
图 4.7 出错数据比特之间的最大距离	53
图 4.8 校验分组失败率分布图	56
图 4.9 校验分组失败率累积分布	56
图 4.10 分组不同大小下, 平均修复单个错误所需要的额外传输冗余	57
图 4.11 添加分组校验规则后过滤器的性能	59
图 4.12 使用分组校验后纠错策略的性能	61
图 4.13 基于GS-EEC的自动重传请求策略	62

图 4.14 不同数据包修复策略的平均重传次数 65

图 4.15 不同出错包修复策略平均冗余量 66

图 A.1 WiFi访问日志收集的真实场景 73

第一章 绪论

1.1 研究背景

无线网络技术是一种便利的数据传输技术，人们可随时、随地的通过无线网络访问互联网资源。无线网络技术在推动网络普及和发展的同时，也在极大的改变着人们的生活方式。以智能手机终端为主要设备的无线网络设备市场发展势头迅猛，相关的市场分析报告[1]中显示，预计2015年，全球智能手机出货量将达到9.09 亿台。这给无线网络的发展注入了强大的动力。随着无线网络的快速发展，人们对无线网络的速度要求也越来越高。近些年，无线Mesh 网络已经被广泛的部署在城市甚至于农村，由此支持远程教育和远程医疗等各种应用中大量的实时流媒体服务。但由于无线信道本身所具有的不可靠的特性，使得无线网络传输不及有线传输稳定。在无线网络中，数据在传输过程中往往因为干扰和低信噪比，使得接收端无法正确解码。对无线网络持续增长的需求也使无线信道更加拥挤。然而在无线网络中，即使收到的数据包出错，错误比特的数量也非常有限，数据包中的大部分比特仍然是正确的。

在无线Mesh 网络中，通常使用前向纠错的解决方案：源端向原始的数据中添加纠错码，以保证数据在目的端正确解码，并避免由于重传而造成的延时。中间节点简单的将所有编码后的数据包按照需要转发，在纠错码帮助下，目的节点能够对出错的数据包正确解码。但是由于无线网络链路特点，这种前向纠错的方案并不现实，因为我们始终无法确保足够的冗余量以保证一次传输的正确；另一方面，高概率的正确传输需要大量的冗余，耗费计算资源和网络资源。

无线局域网的差错控制方案与有线局域网的差错控制方案类似。无线局域网使用自动重传请求纠正传输过程中的错误。这种解决方案也存在问题：在有线网络的可靠信道中传输数据不易产生错误，由于网络拥塞产生丢包则更为常见。这种情况下，使用自动重传请求重传整个数据包作为差错控制的策略非常有效。这种特征与无线网络的信道传输有很大不同。无线网络中更容易收到出错的数据包，同时数据包中的错误数量非常少[2]。因此在无线网络中，由于传输出现少量错误就重传整个数据包，会浪费宝贵的无线带宽资源。

新近研究尝试使用无线网络中的出错数据包，他们认为即使数据包因为干扰等原因出错，这种数据包也包含很多有用的信息。一个典型的例子是专为紧急情况(如对森林大火、洪水或者地震)所设计的无线传感器网络。在该网络中，

系统需要尽可能多, 尽可能快的向控制中心发回数据。对于图片信息而言, 出错的数据包中仍然保留着有用的信息。收集出错的数据包, 能够在最大程度上获取紧急场景的情况, 供控制中心做出决策。文献[2-4]提出, 通过部分重传的方法代替完全重传整个数据包的方法, 修复出错的数据包, 可以减少重传的数据量。这些不同的应用说明出错的数据包仍然是有用的。

受到上述应用的启发, 文献[5]中提出一种新的编码思想, 即编码并不类似前向纠错码那样准确纠正数据包中出现的错误, 而是估算数据包中的错误的数量——误码率。误码率的信息在许多应用中是非常重要的数据。例如在紧急场景中可以通过计算误码率的大小判断数据包携带的信息量出错的严重程度, 从而帮助传感器优先传输误码率较小的数据包, 从而使得基地在有限的时间内收到的数据最大化。在对WiFi 的速率进行调整的应用中, 发送端可以根据接收端反馈的误码率的大小调节发送速率, 从而更有效的利用链路。在多跳的无线网络场景中, 中继节点可以根据收到的误码率大小判断是否要求前一个节点重传数据包: 如果误码率太大, 即使目的节点收到了数据包, 也无法使用, 只能请求重传, 这样耗费了更长的重传延时, 因此由中间节点根据误码率决定是否重传出错数据包, 将降低多跳无线网络中的延时。

1.2 问题分析

估错码在文献[5]中最早被提出, 并引起了研究人员的极大兴趣。文献[5]中给出了估错码编码的目的和可用性的讨论, 并给出了一种有效的编码解码方案和估错算法, 还讨论了EEC 的应用, 例如基于估错码的WiFi 速率调整方案和基于估错码的视频流传输方案等。文献[5]给出的GS-EEC 是基于分组奇偶校验的方式编码的, 一个校验位负责校验一组数据位, 每个被校验的分组大小不同, 分组大小从2比特指数递增, 直到整个数据包成为一个分组被校验。不同大小的校验分组的校验位, 校验不通过的概率对误码率计算的准确度不同。因此要估算数据包中的误码率, 只需要找出最能准确计算出误码率的某些校验位, 并计算出这些校验位的失败率, 就可以估算出数据包的误码率了。

自GS-EEC 提出后, 先后有多名学者对估错码进行深入的讨论。文献[6]提出了新的一种编码方案RAK-EEC, 该编码方案是基于随机行走的策略对数据包中进行计算若干次hash 值, 接收端通过这些hash 值计算得到数据包的误码率。其最大的特点是冗余量和解码复杂度与GS-EEC 处在同一水平, 即信息冗余为常量级别, 解码复杂度与数据包长度呈线性关系。同时, 该编码在误码率增加情况下, 其准确度的衰减比GS-EEC 的性能更加优秀。实验表明RAK-EEC 相比于GS-EEC 能够获得更加稳定的估错精度, 其估算的误码率的均方误 (mean

square error) 变化更加平稳, 其估计的偏差也更加小。

文献[7]给出了一种更为简单有效的估错码编码方案, 从通信复杂度角度建立了估错码的模型, 分析估错码所需要的冗余的下界。同时给出了另外一种基于hash函数的编码方案, 称为tug-of-war (EToW) 方法。实验和理论分析显示, EToW 不仅仅该编码方案冗余更少, 同时, 相比于GS-EEC 估错的性能更为优良。

文献[8]从信息论的角度对GS-EEC 和EToW 进行分析, 给出了GS-EEC 的估错算法的方差, 发现GS-EEC 的精度远没有达到理论的界, 因此给出了基于极大似然的新估错算法, 使得GS-EEC 的估错能力接近理论的界。该文献提出了一种称之为Generalized-EEC(gEEC) 的估错码统一的框架。在该框架下, GS-EEC 和EToW 两种估错码是在其参数不同的情况下的两种特例。最后, 文献还为gEEC 寻找到了—种通用的解码算法即基于Jeffreys prior 的最大似然估计算法 (MLE)。该算法使得EToW 的解码和估错性能都接近于最优。

在文献[9, 10]中引用了一种更为简单的估错码的编码方案: Amplified Sampling(AMPS), 该估错码是估算数据包中的字节错误率的。AMPS 并不需要准确判断出数据包中的字节错误的比例, 而是将数据包分成若干段, 通过计算每一段的错误字节数, 并将计算得到的错误字节数放大, 使得在最大后验概率下能够保证, 放大的错误字节数是大于或者等于真实的最大的错误字节数。通过这种方法, 这种放大的错误字节数可以使得系统重传足够的纠错码修复错误。因此, 相比于GS-EEC, 在字节出错数目很小的情况下, 使用AMPS估算错误数目更适合文献中提及的场景。

这些理论工作基本集中在两个方面:

- 对现有编码做修改, 使估错算法的精度达到理论上的最优;
- 提出新的估错编码方案, 使之能够在某个指标 (冗余度或者精度) 上, 性能优于已知的估错编码方案。

无论是对已知方案的优化, 或是提出新的编码方案, 现有的所有工作都没有讨论估错码是否具有某种程度的纠错能力, 以及纠错能力的强弱。更没有提出具体的新的应用, 对估错码的研究仍然停留在对方案的理论研究和估错编码估错能力的研究上。

1.3 本文主要的工作

由于估错码被用来估算在无线传输过程中数据包的误码率, 相比于前向纠错码而言, 估错码能够保持很低的冗余和很小的计算量。估错和纠错的两种不

同的编码技术，估错码是在编码代价和编码功能方面的折中与权衡的结果。纠错码利用编码代价换取强大的纠错性能。只要信道中的信噪比在香农公式的范围内，就能在理论上正确解码；估错码通过弱化其功能降低了编码的代价，向数据包中添加的冗余量甚至低于循环冗余校验的冗余量，将计算复杂度降低至 $\log n$ 数量级。现有的估错码只能估算数据包中的错误比特数目，不能确定出错的位置，更不能纠正出错的比特。本文研究基于估错码的纠错问题。具体问题是：当利用EEC检测到误码时，能否在不重传整个数据包的情况下，纠正该数据包的出错数据位。

我们首先分析了估错码纠错的能力。研究显示估错码无法修复所有的错误，但却能够有效降低出错的比特数目。我们给出了一种基于估错码（GS-EEC）的纠错策略，该策略充分利用估错码中的校验信息，提出了先过滤后随机翻转的纠错方案。修复策略是基于以下事实：在估错码的编码阶段，任何一个数据位都会被多个校验位校验。基于不同层的校验位的校验结果，我们给出了两种过滤规则，过滤器算法使用这两种规则将正确的数据位分离出来，以很高的概率得到可疑比特集合——一种包含绝大多数出错数据位的集合。通常情况下，可疑比特集合远远小于原始数据包的大小。我们将整个纠错过程转变为一种优化问题，在可疑比特集合中，随机翻转算法通过随机翻转可疑数据位，不断获得使评分函数最小的可行解。理论分析显示我们的策略能在很大概率上使假阳性保持在很小的范围之内，同时能够修复绝大多数出错的数据位。

我们使用真实的数据包对错误纠正策略做了评估。提出的基于估错码的纠错技术拥有广泛的应用。该技术作为对估错码的补充，可提升通信的质量，特别适合错误容忍的应用中。例如，实时的流媒体的应用可容忍错误的发生，错误纠正策略能在不需要重传的前提下流媒体服务质量。该策略同时能够对新的传输技术提供帮助，例如基于误码率感知的数据包调度和重传技术在误码率超过某一阈值时选择重传，使用错误纠正策略能够降低数据包的误码率，可以降低重传的数据量。

我们将估错码的错误纠正策略应用到出错数据包的修复的场景中。首先讨论当前无线网络的数据链路层的差错控制协议：自动重传请求协议。该协议只使用正确的数据包，如果接收到的数据包中存在错误比特，无论错误数量多或者少，该数据包都将被丢弃，接收端将请求发送端重传出错的数据包。这种只接收完全正确数据包的策略影响网络的吞吐率。在启动自动重传请求的过程中，接收端不得不至少等待两倍长的传输时延和一个DIFS一个SIFS以及一个ACK超时时长之后，才能通知发送端请求重传。由于无线网络信道非常不可靠，错误发生的概率很高，因此，由重传请求导致的时延较大。通过深入探究数据包中

错误的规律，我们发现两个很重要的特性：

- 数据包中的错误数量作为随机变量，服从幂律分布；
- 数据包中的错误往往属于突发错误。

基于上述特性，我们将估错码应用到无线网络中，设计一种出错数据包修复技术：EEC-PPR。该出错数据包的修复技术使用GS-EEC对整个数据包进行编码，然后对数据包再分组编码。该协议与传统的自动重传请求不同点在于，充分使用估错码估算数据包的误码率，利用估错码的纠错算法，在误码率极小的情况下自动纠正数据包中的错误，从而避免了重传，降低了重传导致的时延；当误码率较大时，通过分组校验码确定错误的位置，并采用分组重传的方式纠正出错的数据包，避免重传整个数据包造成网络资源的浪费。这种分组重传的方法虽然需要两倍的传输时间，但是由于接收端需要NACK通知发送端，因此相比于传统的自动重传请求，减少了接收端等待ACK超时所用的时间，在一定程度上降低了重传需要的时延。因为数据包中的错误数量服从幂律分布，大多数的数据包中包含的错误数量非常少，这使得大部分数据包中的错误能够直接被估错码纠正而不需要重传；由于数据包中的错误属于突发错误，这使得使用基于分组重传时，只需要重传很短的几个分组，就能够纠正所有的错误。我们基于冗余量最小的原则，对分组大小进行了讨论。同时对分组校验码的数量进行了讨论，校验码数量愈多，则分组校验码提供的出错比特位置就越准确，本文分析了两者之间的数量关系，然后在可接受的误差范围内确定了分组校验码的数量。

在对原始数据包增加了分组校验码后，为了能够充分利用这些校验信息纠正出错比特，我们对数据包的错误纠正策略做了进一步修改。首先在最优化目标函数中增加了分组校验码的成分，结合估错码校验位和分组校验位判定选取的比特组合是否是真正的错误比特。其次在过滤器算法中添加新的规则，该规则利用分组校验判断出错比特所在的位置，使得过滤器的性能得到了极大的提高。

我们对比了EEC-PPR和文献[4]中提出的Maranello以及文献[3]中提出的ZipTx两种技术，并在真实的WiFi访问日志中做模拟。结果显示，我们提出的方案能够有效的减少数据包的重传次数，降低编码冗余，提升了网络的性能。

本文的主要贡献可以概括为如下几点：

- 对基于估错码的纠错问题进行形式化。使用估错码的错误修复技术在本文中被形式化为优化问题。给定接收端的数据包和纠错码的校验位，本文定义了一个优化目标函数，使得出错的数据位最少。
- 提出了一种过滤器算法。本文所提出的过滤器算法，通过利用估错码的校验位的信息，以区分在传输过程中正确的数据位和出错的数据位。该算法得到包含有绝大多数出错数据位的可疑位集合，为问题的解降低了搜索的空间。理论分析给出了该算法导致假阳性的上界。
- 提出了一种随机错误纠正算法。本文提出了一种随机翻转算法，用以纠正出错的数据位。该算法通过随机翻转可疑数据位，测试多种可行解，以最小化优化目标。当误码率足够小时，可以证明，该算法能够在很大概率上纠正绝大部分错误的的数据位。
- 针对无线网络，设计了一种新的出错数据包修复协议EEC-PPR，该协议充分利用GS-EEC的估错和纠错能力，有效减少数据的重传次数和重传数据量，减少数据包的编码冗余，提升了网络的性能。
- 基于真实的WiFi访问日志进行性能评估。利用真实WiFi访问日志，我们评估了本文中提出的策略的性能，结果显示该策略有效的降低了数据包中的误码率，降低了系统时延，减少了编码冗余。

1.4 本文的组织结构

本文将围绕估错码，特别是基于GS-EEC编码的纠错问题展开讨论，其余章节安排如下：

第二章介绍研究背景和相关工作。首先从信道编码与差错控制开始，我们回顾了经典的差错控制编码的不同方案，包括不同检错码的设计和各种纠错码的编码解码方案，并对它们的不同做了简单的分析和比较。对差错控制的另外一种解决方案——自动重传请求做了较为简单的介绍。通过对上述差错控制方案的说明，从而引出本文讨论的估错码。在介绍估错码的过程中，本文介绍了估错码的发展过程、应用场景，对现有的估错码的工作做了较为深入的讨论，并对比了不同工作的特点。最后引出本文对估错码的研究工作。由于我们将估错码的错误纠正技术应用到出错数据包的修复问题中。因此在研究背景中，本文还讨论了现行的出错数据包的修复技术，为第四章应用做铺垫。

第三章介绍估错码的错误纠正技术。首先给出主要目标：研究估错码的纠错技术，增强估错码对差错控制的能力。3.1节形式化的将估错码纠错问题转变

为最小化问题，3.2节出了错误的修复方案。该修复方案包含两个部分，即过滤器和随机翻转算法。3.3节给出了对过滤器的详细设计和性能评估，3.4节中给出对随机翻转算法的描述和复杂度分析。针对提出的纠错策略，在??节使用真实数据集对提出的策略进行评估，在最后给出了评估结果。

第三章给出了估错码的错误纠正策略。第四章将错误纠正策略应用到出错数据包修复问题中，提高网络的性能。本文在4.1节的内容中描述了出错数据包修复的问题。在该场景中，估错码具有明显的优势。4.2.2节基于统计的方法，给出了无线信道中的错误分布特点，包括数据包中误码率的分布，和在同一个数据包中错误比特位置的分布特点。其次，在4.3节结合这些信道特点，将估错码应用到对数据包的编码中，同时基于分组校验确定出错比特的位置。通过上述的设计，本章给出了一种新的出错数据包的修复方案。在4.4节，我们对这种修复方案的纠错进行分析，发现这种编码方案提升了估错码的纠错能力，通过与现行主要的两种修复方案对比，发现这种修复方案在两个主要的修复指标上都有较大的性能提升。

在第五章，我们给出了对工作的主要内容做了总结和分析，提出了工作中的不足，并展望未来，讨论将来如何加强和改进工作的缺点。

第二章 相关工作

本章将讨论与估错码相关的科研成果。首先介绍信道编码的概念，并介绍几种著名的编码技术及其应用，然后介绍估错编码的概念和发展过程，最后介绍对含有错误的数据包的修复工作的进展。

2.1 信道编码及差错控制

在数据通信的过程中，由于闪电等自然因素或者不同信道之间的干扰等人为因素，使得信道传输不可靠，往往导致接收端收到的信息与发送端发送的信息不一致。为了提高信道的可靠传输，降低甚至消除数据在传输过程中的错误，提高传输过程中的抗噪性能，则在传输过程中，对错误的控制变得非常重要。纠错码就是这样一种对差错进行控制的信道编码技术，旨在控制数字传输过程中的差错。一般而言，根据信道编码中的功能不同，可以分为纠错码和检错码两种类型。纠错码不仅能够发现在传输过程中的错误，同时能够在一定程度上纠正这些错误，使得接收端能够收到与发送端相同的数据。检错码则仅仅只能通过计算发现传输过程中的错误。然而，相比检错码而言，纠错码需要大量的冗余信息和复杂度很大的编码解码算法的支持。

2.1.1 检错码

检错码是具有发现错误能力的一种信道编码。在使用检错码的通信系统需要向发送端反馈结果，如果检测出错误，则将通知发送端采取相应的补救措施。重复码、奇偶校验码，校验和等都是检错码的重要方法。

最基本的检错码当属于重复码，该编码简单的将原始的数据重复若干次。如果重复的码字在接收端不相同，则认为错误发生了。如果错误仅仅影响了一个码字的少部分，则重复码不但可以发现错误，甚至能够纠正这样的错误。不难看出，重复码的编码效率非常低下。奇偶校验码是接收端通过验证集合中的比特有奇数个“1”或偶数个“1”来判断在传输过程中发生了错误。奇偶校验计算速度快，相比重复码，编码效率也大大提高，但是也存在着若干问题，例如：如果集合中同时有偶数个比特发生错误，简单的奇偶校验就无法判断。校验和是对一个长度固定的数据，通过算术模运算求得的值。接收端通过检测这种校验和判断收到的数据是否正确接收。这种方法被广泛的使用在对文件的存储和读取等应用当中。循环冗余校验[11]是被广泛使用一种校验和方法，是能

够检测出单比特错误的循环码，该种编码对固定长度的数据做移位后，并与生成多项式做模2除运算，所得到的余数成为是否正确传输的依据。循环冗余校验不能可靠检验数据完整性[12]，且其检测错误的能力依赖于生成多项式的长度[13]，但其对突发错误具有较好的检测能力。伯杰码[14]可以检测任意数目的单向错误的检错码，在非对称信道中，从“1”跳变到“0”的概率和从“0”跳变到“1”的概率大有不同，伯杰码通过计算信息中“1”的个数来实现检测错误数量的功能。这种编码在一些应用场景中非常有效，例如计算机内存中，更有可能从“1”衰变为“0”。纠错码也可以被用作检查错误。例如经过纠错码编码后，若接收端收到的码字不属于合法码字，则可以认定为出现了错误。若假设基于汉明距的纠错码的最小汉明距为 d ，则该编码能够检测出码字中 $d - 1$ 位的错误。

2.1.2 纠错码

纠错码或前向纠错码在不可信信道传输中，是一种非常有用的错误控制技术。其基本思想是发送端对要发送的信息增加一些编码冗余。接收端如果检测出收到的数据不合法，则可以通过添加的冗余修复错误并获得正确的编码，而不需要通过发送端重传的方式获取正确信息。纠错码使用在重传代价较大或者不可能重传的场景中，例如单向通信连接和具有多个接收端的多播场景中。

纠错码从形式上可以分为两种类型：分组码和卷积码。

分组码在纠错码中占有重要的地位，大量不同的编码可以归为分组码。分组码的基本思想是将等待发送的数据按照独立的分组进行处理和编码。每个长度为 k 的分组，经过编码后长度为 n 位的二进制码，称为码字，分组码一般用 (n, k) 表示。典型的分组码有汉明码[15]，里德-所罗门码[16]和格雷码[17]。

汉明码的检错和纠错能力取决于编码系统中的最小汉明距。两个码字之间汉明距是指其对应的位置上不同的字符的数目。最小汉明距指在该编码系统中，所有的码字两两汉明距的最小值。若某一个编码系统的最小汉明距为 d ，则该编码系统能够检测出 $d - 1$ 个错误，能够纠正不大于 $\frac{d-1}{2}$ 的错误。1950年，Hamming在文献[15]中给出了 $(7, 4)$ 这种汉明编码方案。在该编码中4位原始数据比特和所增加的3位奇偶校验位，组成了长度为7位的码字。该码字可以检测所有的单比特错误，和部分双比特错误。

里德-所罗门码是1960年被Reed和Solomon提出的一种线性的分组码。里德-所罗门码最大的特点是在随机错误和突发错误的纠错方面性能非常优越。里德-所罗门码是被证明的非常好的检错和纠错编码[18]。给定码字取自 $GF(q)$ 的具有能够纠正 t 位错误比特的里德-所罗门码，则该码字长度为 $q - 1$ ；所拥有的

奇偶检验的位置为 $2t$ ，所具有的且其最小的距离恰为 $2t + 1$ 。

格雷码又称作二进制格雷码，由Frank Gray在1940年提出，常用于模拟信号到数字信号转换的通信系统中。格雷码是一种循环码，二进制格雷码最大的特点是任意两个相邻的码字之间只有一个二进制数不同，例如，传统的二进制数中，3被表示为011，当3变为4时，二进制转变为100，需要跳变三位。而正是这种多个二进制位的跳变，导致了许多可能的中间状态的存在，如010，101等等。更多中间状态意味着更大可能性出现错误。格雷码正是在编码的相邻步进的场景中，变更最少的二进制数位，从而提高转变的可靠性。

低密度奇偶校验码由Gallager在其博士论文[19]中首次提出的，具有稀疏校验矩阵的线性分组码。Tanner从图论的角度重新解释了低密度奇偶校验码。Kou等人基于有限几何理论提出了一种系统的构造低密度奇偶校验码的构造方法[20]。MacKay等对低密度奇偶校验码提出了可行的译码算法[21]，使得后来人们发现了该编码的优良的性能。低密度奇偶校验码的误码率能够非常接近香农极限。该种编码具有很好的分组译码性能，译码不基于网格[22]，译码复杂度低。正是基于这些优点，低密度奇偶校验码成为了高可靠通信系统的备选方案。例如在4G网络通信技术中，该编码成为强有力的竞争者，并且已经广泛应用于光纤通信、卫星数字视频和音频广播等领域。

由于低密度奇偶校验码的校验矩阵中，“1”的数量远远少于“0”的数量，即“1”的密度低，并且编码越长，密度越低，因此而得名。这样做的目的是为了使得解码过程可行。对于低密度奇偶校验码而言，先后提出了三类解码算法：硬判决解码，软判决解码和混合解码。硬判决解码过程为：解码器将收到的模拟信号通过解调器进行解，根据输出信号，做有限的N比特量化，如果高于某个阈值，则认为是1，如果低于某个阈值，则认为是0，由于硬判决会损失信道的信息，导致信道信息没有被充分利用。硬判决解码过程的复杂度低，但同时得到的误码率的性能一般，低密度奇偶校验码的硬判决算法主要包含一步大数逻辑算法和比特翻转算法。

一步大数逻辑算法是基于简单的思路：由于低密度奇偶校验码的每一个数据比特位置 l 都被正交的校验过 γ 次(γ 是编码的参数)，则当 l 存在 $\lfloor \frac{\gamma}{2} \rfloor$ 或者更少的错误时，即可以判定该比特位置正确。比特翻转算法是由Gallager提出来的。其基本原理是：如果接收端的某个比特位置有错误，且这个错误被检测出来，则校验矩阵中，一定存在奇偶校验不能通过的地方，如果任意翻转某些比特位置，则会改变奇偶校验不通过的个数。具体做法如下：接收端计算所有的奇偶校验结果，翻转满足某种条件的某一比特位置的值，并重新计算奇偶校验的结果，重复迭代这个过程。直到所有的奇偶校验的值都通过。这样，得到的修正后的

数据就应该是合法的码字。关于如何选取要翻转的比特的位置，Gallager给出了基于阈值的查找方法，即如果某个比特位置上，奇偶校验的错误数大于某个值则翻转，如果小于该阈值，则不翻转。该阈值的选取与编码时的编码参数，码字的最小距离和信道的信噪比有关系，这个阈值的设定将影响纠错的性能和计算量。需要说明的是，比特翻转算法中，阈值具有自适应的特点。如果阈值选取过大，则可能导致纠错过程失败，这时，就减小阈值继续解码纠错过程。如果阈值选取过小，则可能造成计算量过大等问题。如果阈值选取合适，且误码率较低，则比特翻转算法能够在经过很少的迭代后完成纠错过程。在误码率超过纠错能力的情况下，比特翻转算法甚至也能纠正这些错误。

软判决算法具有代表性的主要有最大后验概率算法和迭代的基于置信度传播的算法。

最大后验概率算法是由Bahl, Cocke, Jelinek和Raviv等人在文献[23]提出的一种基于软输入和软输出端解码纠错算法。该算法最初是设计用来最小化误码率，并提供解码可靠性大小分析的，其核心思想被用在Turbo码解码中，也被引入到低密度奇偶校验码的解码过程中。其基本思想是，通过计算其中的一个比特位置的概率，随后以该概率为基础，计算下一个比特位置取值；的概率，迭代直到所有的比特位置的取值使得其概率接近于1即可。最大后验概率的解码纠错性能非常优秀，但是其计算复杂度非常大，因此出现了简化的最大后验概率的算法，有效降低其计算量。

基于置信度传播的迭代算法也通常被称为和积算法，是一种已知性能最好的解码算法。该算法在Gallager的博士论文中提出。该算法基于独立性假设，利用所有的数据位的信道信息的观察值作为参数，在每一次迭代过程中，不同数据位置之间传递概率信息。该算法将原始数据位置称为变量节点，将校验位称为校验节点。置信传播过程中，变量节点先向校验节点传递一些关于其取值的概率信息，这些信息取决于经过上一轮迭代后，该节点的概率值以及该节点在信道中的观察值。校验节点传递给变量节点的概率信息，是通过与该校验节点相连的其他的校验节点的在上一轮的迭代过程中传递给该节点的值。经过形式化后可以发现，迭代的置信度传播算法可以用两个复杂的公式表示，第一个公式主要采用求和的方法，第二个公式采用乘积的方法，因此，该方法也被称为和积算法。

由于软判决算法充分使用了信道的信息，并且依据了大量的计算，因此，软判决算法的解码纠错性能最佳。为了能够像软判决算法那样充分使用信道信息，同时降低计算复杂度，有研究者就设计出二者混合的解码算法，即混合算法。混合算法主要包括了加权比特翻转算法和加权的大数逻辑解码算法。这两

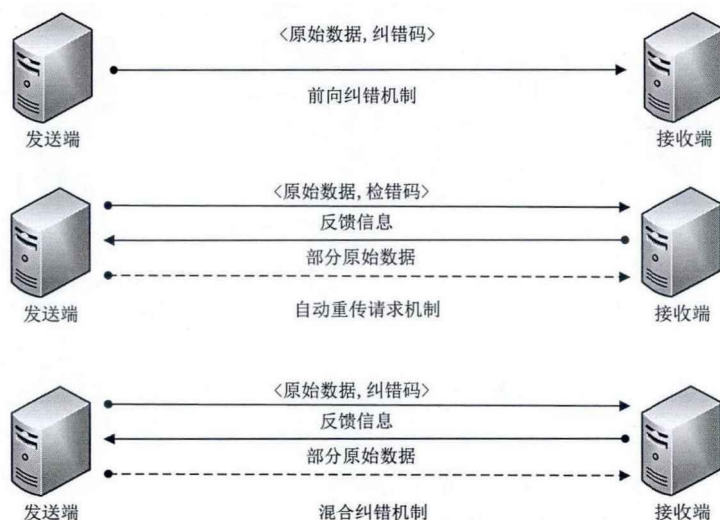


图 2.1: 三种常用的差错控制方式[24]

种算法都是将软判决中信道对接收的每一位比特位置的观察值，也称为置信度作为权值应用到硬判决的两种算法中。对于加权的大数逻辑解码算法而言，是否判定某一个比特位置出错，取决于经过加权计算后的该位置上的校验和是否大多数通过。加权的比特翻转算法也与之类似。

卷积码是相对于分组码而言的另一种信道编码技术，卷积码能够保持信道的记忆效应。在编码阶段，输入的原始信息将与编码器的脉冲响应做卷积运算，故而得名卷积码。卷积码和分组码的有着很大的不同，即卷积码没有将原始数据先分组后编码，而是将连续输入的数据得到持续的编码得到连续输出的编码序列。分组码在编码阶段，某一分组中的所添加的 $n - k$ 比特校验位只与当前的分组中的 k 比特原始数据位有关，与其他分组的数据位没有关系；卷积码的编码器在对 k 比特原始数据位编码为 n 比特的数据位时，这 n 比特编码后的数据位不但与当前的 k 比特数据位有关，而且与当前 k 比特之前的 $m-1$ 段数据有关。 m 为编码器的存储级数，即在编码器中，同时存在 m 个数据块，这 m 个数据块共同编码形成编码流输出。卷积码与分组码的另一个重要的不同点在于，卷积码并不是通过增加 n 和 k ，而是通过增加级联的存储级数 m ，来实现大的最小距离和降低误码率的[22]。

卷积码[25]自1955年时被提出后，各种不同的译码算法被提了出来，有序列译码[26]，门限译码[27]以及后来的维特比最大似然译码算法[28]和最大后验概率译码算法[23]等。卷积码的编码器分为前馈编码器和反馈编码器。1993年文献[29]中Berro和Glavieux等人利用两个反馈编码器的并联，结合最大后验概率译码算法，提出了新的编码方案即Turbo码。Turbo码能在性噪比接近香农公

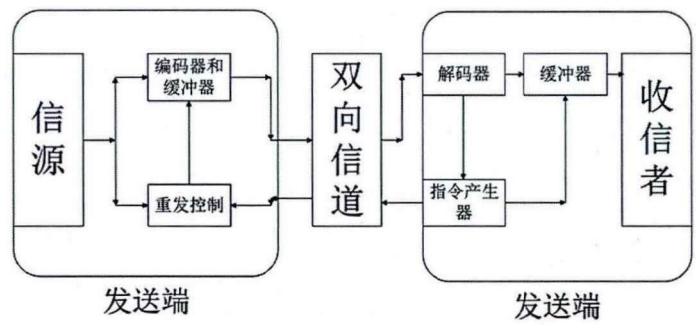


图 2.2: 自动重传请求流程图[24]

式所给的极限时，实现相当低的译码误码率。卷积码被广泛的应用于各种应用场景中，包括移动通信和卫星通信当中[30]。

检错码使用能够检测错误是否发生，但却不能反映错误的严重程度，无法获得误码率大小。误码率大小在许多场景中是非常重要的信息。纠错码不但能够发现错误，同时能够纠正错误，但是纠错码向原始信息中添加了较多冗余，其编码解码因为计算复杂而往往需要硬件的支持。在一些场景中，我们需要一种能够获得误码率，又能纠正错误，计算简单，冗余量少的编码形式。对于检错码和纠错而言，都不适合。这正是估错码所具备的优势。

2.1.3 差错控制

通过使用不同的信道编码和修复策略，在数据通信过程中，存在三种常用的差错控制方式，即自动请求重传(ARQ)，前向纠错(FEC)和混合方式(HEC)，如图2.1所示。

前向纠错系统中，发送端将原始数据编码成为可以自我修复的纠错码字，如果在数据传输过程中出现错误，则接收端就能够使用纠错码发现并纠正错误。前向纠错的这种差错控制方式由于使用了比较多的冗余信息，不仅影响数据传输的效率，同时由于解码和编码过程复杂，设备昂贵，导致计算复杂度高。与检错重发的策略相比，前向纠错具有一定的优势：可以避免重传，有较好的实时性，特别是在单工信道中拥有极为恰当的应用场景。前向纠错的主要性能，如解码误码率，复杂度等都取决于在该策略中使用的纠错码的性能。不同的纠错码拥有不同的特点，具体分析在对纠错码的介绍中已经给出。自动请求重传(ARQ)是原始数据包使用检错码编码，如果在数据传输过程中出现错误，则接收端可以检测出错误发生并反馈错误信息，发送端通过重发数据包纠正错误。这个过程将继续下去，直到数据成功发送或终止条件被触发。需要说明的是，存在这样的可能性：即发送端没有正确检测出数据包中的错误。这种漏检的概

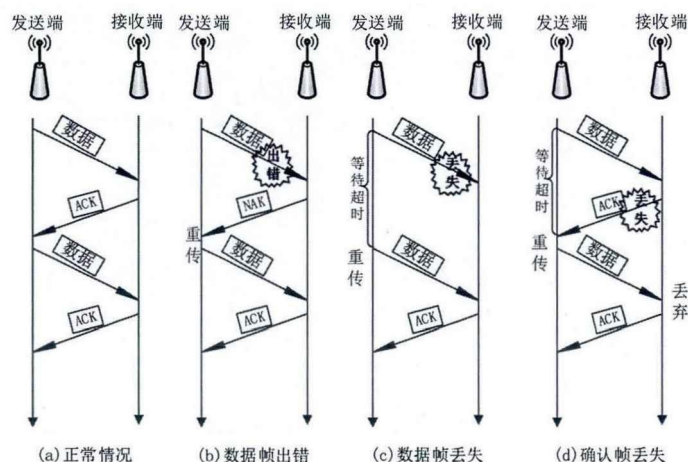


图 2.3: 停止等待ARQ

率与检错码相关。自动重传请求成为OSI七层模型中数据链路层所使用的差错控制协议，通过使用确认和超时机制，以保证在不可靠的底层服务基础上实现可靠的数据通信。通常的自动请求重传有两种基本类型，即等待式自动重传和连续ARQ(连续ARQ包含了两种方案，即退N步重传和选择重传)。图2.2是自动请求重传的流程图。

停止并等待ARQ的工作原理可以描述为：发送端向接收端发送数据帧（数据帧的末尾往往添加了冗余信息，以帮助接收端判断数据帧是否正确），并等待接收端返回确认信息（ACK），启动ACK计时；在等待ACK期间，发送端不再发送新的数据；如果数据帧没有被成功接收（产生丢包或者接收端发现错误）则接收端不发送确认信息。发送端等待直到ACK计时超时后，重新发送副本；该过程一直重复直到接收端正确接收了数据。可以看出，在停止等待ARQ的过程存在两个问题：

- 如果接收端正确接收了数据，并向发送端发送ACK，而确认信息却丢失。则当计时器时间到，发送端就重新发送前面的数据帧副本给接收端，这时，接收端在不知情的情况下，会收到两份相同的数据帧，并且无法判断该帧是不是前一个帧的副本；
- 当传输的时延变得很大，以至于当发送端要超时重传数据帧时，该数据帧还没有到达接收端，这种情况下，接收端将接到数据帧的两个副本。

为了解决上面所提到的副本问题，最简答的处理方案为在每个数据帧上增加序列号。接收端在ACK上标注出希望接收的数据帧的编号。通过序列号的方法

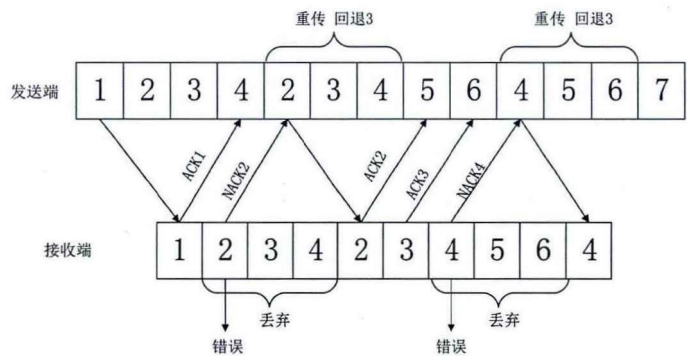


图 2.4: 回退N-ARQ

式，接收端就可以辨别出数据帧是否是副本，发送端也可以正确判断是否需要发送副本。

回退N ARQ 是另一种自动请求重传协议。该协议的不同之处在于，发送端会连续发送若干个数据帧，而不必等待来自接收端的对每一帧的确认。接收端会在它所发送的每个确认帧中说明它等待的下一个数据帧的序号，接收端在接收到数据帧后，判断其序号，如果该序号不是接收端期望的序号，无论是序号表明已经被接收或是表明未接收过，该数据帧都将被丢弃[31]。发送端发出了所有在发送窗口中的N个数据帧后，将等待侦听接收端的ACK，如果发送端收到的NACK表示接收端没有收到某个数据帧，则发送端将回退到该帧的位置，并重新填充发送窗口，开始发送过程。该协议是滑动窗口协议的一个特例：当发送端的滑动窗口大小为N，而接收窗口大小为1时，滑动窗口协议即为即为回退N ARQ。该协议比简单的停止等待ARQ更加有效，因为该协议在本应该停止发送等待确认的时间段，仍然在发送数据帧。但是该协议也使得数据帧的重复发送的次数变多。一旦某个数据帧没有被正确接收，发送端就必须回退并重新发送之前已经发送过的一部分数据帧。为了避免这种情况的发生，选择重传ARQ被设计出来。

选择重传ARQ是一另一种自动重传请求协议。在该协议中，发送端发送位于发送窗口中的数据帧，并且发送端不需要等待接收端对每一个帧确认。接收端不需要像回退N ARQ 那样对若干个数据帧做一次确认。接收端可以乱序的接收数据帧并缓存起来，发送端则将接收端没有正确接收的数据帧重新发送。在选择重传ARQ中接收窗口和发送窗口必须等大小，并且是最大数据帧序号的一半大小，这样做是为了避免当接收端的所有确认都丢失的情况下，发送端重新发送窗口中的数据帧后，接收端无法判断是否是之前接收到的副本的问题。在上述提到的三种自动请求重传的方案中，选择重传ARQ效率最高，但不可否认，

实现起来更为复杂。选择重发ARQ需要全双工的链路，而停止等待ARQ只需要半双工的链路。

混合纠错是一种结合前向纠错和自动重传请求的混合系统。如果错误能够被前向纠错吗纠正,则使用前向纠错，否则，使用重传机制重传该数据包。

2.2 估错码

Chen等人在文献[5]首次提出估错码的概念，编码解码方法以及误码率的估算算法，旨在在无线网络的传输过程中，估算数据包中的误码率（Bit-Error-Rate）。有许多研究工作对文献[5]中所提出的估错码的编码、解码和估错算法进行了分析和优化。估错码优秀的估错精度，简单的编码解码算法，相比纠错码更低的冗余，更低的复杂度，吸引了大量的科研工作者投入到相关的研究当中，目前有研究工作将估错码应用到不同的场景中，解决了各种问题，获得了很好的效果。

2.2.1 估错码的设计理念

在文献[5]中，作者提出了估错码的设计理念：出错的数据包在以下四种情况下仍然具有相当的使用价值。

- 在使用自动请求重传机制的差错控制技术时，接收端请求发送端发送额外的冗余数据，使得之前出错的数据包能够得到修复和使用；
- 接收端可以通过一定的方法通知发送端发送额外的数据，将出错的数据包拼接组合形成完整为出错的数据；
- 在某些场景中，特别是实时的视频流传输过程中，如果使用前向纠错技术，当误码率在一定范围内时，则可能完全纠正这些错误，而不避免重传，降低时延。
- 在可容忍错误数据的场景中，出错的数据包仍然携带着十分有用的信息，可以被系统所使用。

上述不同场景的假设是基于两个非常重要的前提：

- 对估错码的技术指标的要求：对误码率的求取和计算非常简单而且高效，即利用很少的冗余信息，利用简单的编码解码技术，以很低的计算复杂度为代价，就可以获取数据包的误码率；

- 估错码的应用可行性的要求：误码率能够成为非常重要的信息促使在不同场景中提高系统的有效传输的比例。

在上述的场景中，相比使用纠错码的编码技术，获取误码率显得更为重要，例如在使用基于自动重传请求的错误修复策略时，当获得了数据包中的误码率信息，则能够更为高效，准确的传输适量的冗余修复出错的数据包；在使用出错数据包的分片重传策略时，当获得了哪些分片出错，以及误码率大小的相关信息时，更能有效的帮助重传和分片拼接组合的策略获得正确的数据包；在使用前向纠错技术保证实时视频流传输中，如果误码率超过了纠错码的纠错能力，则在中间节点就应该放弃转发，丢弃数据包，并请求重传节省时延，如果数据包中的错误能够被纠错码所纠正，则直接传输到下一个节点，通过这种方式，提高用户的QoS；在某些可容忍错误数据的场景中，如果可以获取每个数据包的误码率的大小，则可以根据场景判断，保留或者丢弃数据包。

事实上，为了获取误码率，在文献[5]中提到估错码的编码方案（简称为GSEEC，因为该编码方案是基于分组抽样的错误估计码，即Group Sampled Error Estimating Code）中所添加到原始数据包中的冗余信息为 $O(\log n)$ ，其中 n 表示数据包的大小，在真实的系统中，仅占数据包大小的2%，如果只估算误码率是否超过了某个阈值，则冗余信息甚至可以达到4字节，如此低的冗余度，甚至超过了循环冗余校验码。为了估算误码率，需要增加的额外计算复杂度为 $O(n)$ 。由此可见，该文献中提出的估算和编码方案符合可以达到前文中对估错码技术指标的要求。就应用可行性而言，估错码估算出数据包的误码率后，可以将数据包的误码率信息使用在不同的节点上，从而达到不同的效果。

发送端使用误码率信息：事实上，数据包的误码率信息是在传输过程中各种策略和媒介性能和参数的综合反应，如调制和编码策略是否合适，信道的带宽大小以及发送过程中能量选择是否合适等等。发送端通过使用误码率，对上述可变因素进行调整，最终达到最大的传输增益。具体而言，

- 通过估错码计算得到的误码率可以作为发送的数据率调整依据。当误码率增大，则说明数据率过大，需要降低发送端的数据率；如果误码率持续为0则，说明数据率较小，可以通过提高发送端的数据率，充分利用信道传输数据。当前主流的设备或协议中，主要是基于数据包的丢包率或者数据包中的信噪比来选择不同的数据率。该文献的实验表明，使用数据包的误码率作为发送端数据率的评价指标，将获得更好的效果。文献[32]和文献[33]中，同时指出，使用误码率同样可以指导发送端对信道，发送功率，甚至天线方向的选择问题上。

- 在多跳的无线传输过程中，基于误码率的路由选择能够获得更加精准的结果，通常意义上路由选择是以正确传输数据包所需要的最小数目作为衡量的标准的，但假设发送端能够得到每个数据包中的误码率的信息，则对不同链路的状态将更加清楚准确，选择的路由的选择也将更加合理。

在接收端，误码率作为重要的信息不仅要作为接收状况的信息向发送端反馈，在多跳网络中的中间节点，数据包的误码率可以用在不同的场景中：

- 基于误码率的数据包重传：在无线传输过程中，特别是在实时流媒体传输过程中，为了降低重传造成的时延，提高QoS，在编码过程中，往往采用前向纠错的方法，在原始的数据中添加大量的冗余信息，保证在目的端，即使数据包中出现错误也能成功纠正。然后如果数据包中出现大量的错误，误码率超过了前向纠错码纠错的能力，则目的端仍然需要重传该数据包。为了避免在目的节点才决定重传的问题，源端可以在数据包的头部添加一个重传的阈值信息，并将数据包用估错码编码。任何收到该数据包的中间节点可以通过估错码计算误码率的大小，并且对比是否超过了纠错码的纠错能力，从而在中间节点就决定是否重传数据包，节省了在目的节点重传所需要的延时。
- 在一些严重危害的监控系统中，例如地震，山洪，火灾的预报与监控系统要求更快更多的将信息传递到目的节点。在这种场景中，即使数据包中包含有错误，仍然携带了有用的信息，因此，当误码率小于某个特定阈值时，允许数据包传递到目的节点更为合理。

从上面的论述中，可以得到估错码具有很好的应用前景，特别是在含有出错了数据包的发送、中继、修正和使用等方面，都有潜在的应用价值。

2.2.2 GS-EEC编码解码方案

文献[5]给出的GS-EEC是基于分组奇偶校验的方式编码的，如图2.5所示。在GS-EEC的编码过程中，一个校验位负责校验一组数据位，每个被校验的分组大小不同。分组大小从2比特指数递增到整个数据包大小。不同大小的校验分组的校验位校验不通过的比例，符合以误码率为参数的一种概率分布。因此要估算数据包中的误码率，只需要利用出最符合的某些校验位，并统计出这些校验位的失败率，就可以估算出数据包的误码率了。由于我们的主要工作是基于GS-EEC编码，因此在本小结中，我们将形式化的描述GS-EEC的编码和解码问题。

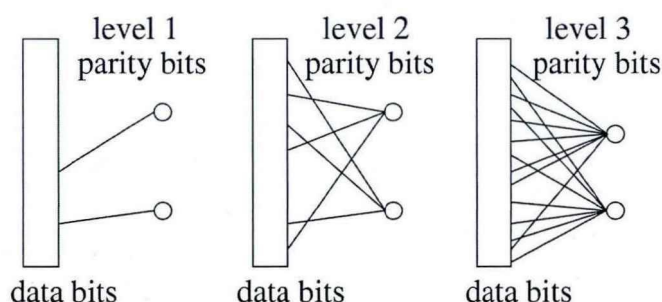


图 2.5: GS-EEC的编码算法示意图[5]

设待编码的数据包的大小为 n ，共有 k 比特校验位参与校验该数据包，根据每个校验的分组大小的不同分层，每层有 s 个校验位，共有 l 层。每个校验位随机从整个数据包中选择数据比特做奇偶校验。设第 i 层的校验分组大小为 $2^i - 1$ ，即第 i 层的每个校验位都随机从数据包中有放回的任意选择 $2^i - 1$ 个数据位做校验。并最终生成所有的校验位的校验信息。编码的第二步是所有的数据位和所有的校验位随机选择新的位置，并最终组成新的数据包。GS-EEC为了能够避免任何一种可能的出错分布影响对误码率的估算，故采用了两次随机的过程。第一步每个校验位校验哪些数据位校验是完全随机生成的；第二步是校验结束后，要传输的数据包中的比特的排列和原始的数据包的比特位置不同，是随机生成的，特别的，校验位也完全随机插入到新的数据包中。图2.5中展示了GS-EEC中前三层的校验位是如何校验数据位的，其中每层的校验位 $s = 2$ 。

通常而言，一个校验位校验一个分组中的 g 比特数据位，该校验位只能告诉我们所校验的分组中出错比特数目为奇数还是偶数。如果奇数个错误发生，则该分组的校验不通过，如果有偶数个错误发生，或没有错误发生，则该校验位在接收端将通过校验。当误码率和分组大小满足一定的条件时，校验位的校验结果就能告诉我们更多的信息。例如：当 $p_0 \leq \frac{1}{2g}$ 时，如果该校验位通过校验，则有很大概率证明该分组中的所有数据位都是正确的，如果该校验为没有通过校验，则可以证明该校验分组中只包含一个错误的概率非常大。简单的讲，当误码率在足够小的情况下，如果 s 位校验位中校验出错的占 s 位校验位的比例为 q ，则我们可以认为 sq 个分组中恰好有 sq 为数据位出错，于是，整个数据包中的误码率也可以计算得到。在GS-EEC中，将校验分组大小设计为多层次，使得在所有的 l 层的校验位中，当数据包的误码率小于0.25时，总有一层的分组大小满足 $p_0 \leq \frac{1}{2g}$ 的条件，因此可以充分利用满足条件的这一层的校验位，计算得到数据包的误码率。在估算误码率时将更为复杂，但基本的思想如上面所述。

在文献[5]中, 有充分的理论分析保证, GS-EEC的估算精度可以使用形式化 $\varepsilon - \delta$ 语言描述如下:

任何给定正常数 ε 和 δ , 当 n 足够大时, 存在 $s = O(1)$, 使得若使用 s 作为GS-EEC编码的参数 (同时对于输入的两个参数 a 和 b , 其中 $\frac{1}{n} \leq a \leq b \leq \frac{1}{4}$), 则在GS-EEC的编码中下列的事件发生的概率至少为 $1 - \delta$

- 如果 $p \in [a, b]$, 则对 p 的估算值 $\hat{p}$ 所处的范围为 $\hat{p} \in [(1 - \varepsilon)p, (1 + \varepsilon)p]$;
- 如果 $p < a$, 则对 p 的估算值 $\hat{p}$ 所处的范围为 $\hat{p} \leq (1 + \varepsilon)a$;
- 如果 $p > b$, 则对 p 的估算值 $\hat{p}$ 所处的范围为 $\hat{p} \leq (1 - \varepsilon)b$ 。

(其中 a 和 b 是两个与估错码相关的参数)

可以证明, 该编码方案的编码, 解码和估错的复杂度都是 $O(n)$ 。GS-EEC是最先被提出的估错码, 其特点是编码简单, 冗余度低, 计算复杂度低, 且理论证明充分。在该编码被提出后, 先后有不同的估错码的编码和估错被提出, 它们各有特点。

2.2.3 RAK-EEC编码解码方案

在文献[6]中, Yao等人提出了新的一种编码方案, 该编码方案是基于随机行走的策略编码。其最大的特点是冗余量和解码复杂度与GS-EEC处在同一水平, 即信息冗余为常量级别, 解码复杂度与数据包长度呈线性关系。该编码在误码率增加情况下, 其准确度的衰减比GS-EEC的性能更加优秀。

简单的讲, RAK-EEC使用 m 组hash函数作为判断是否出错以及错误数量的根据。 m 是与数据包长度 n 无关的常数。每一个hash函数的计算过程可以用一个长度为 n 的向量和原始数据的内积表示, 用 R_i 表示第 i 个hash函数, 用 D 表示原始的数据包, 则第 i 个hash函数值的计算结果为: $S_i = \langle R_i, D \rangle = [1, -1, \dots, -1, 1][1, 0, \dots, 0, 1]^T$, 图2.6为 m 组hash函数值的计算示意图。

其中 R_i 的每一个分量只有两种值, 1和-1, 每个分量的取值是随机变量, 独立同分布。在发送端随机计算出每一个hash函数的计算向量 R_i , 并计算得到每个hash函数的值 S_i 。发送端发送的数据包为 $\langle D, S_i, Seed \rangle$, 即包括了所有 m 组hash值, 随机种子和原始的数据包, 由于hash值和随机种子对一次传输非常重要, 因此为了保证一次传输正确, hash值和随机种子使用纠错码编码, 保证一次正确传输。

在接收端, 发送端将重新根据随机种子获取用于 m 组向量 R , 并通过这些向量, 和收到的数据 D' 计算得到新的 m 组hash函数值 S'_i 。通过对比 $S_i, i =$

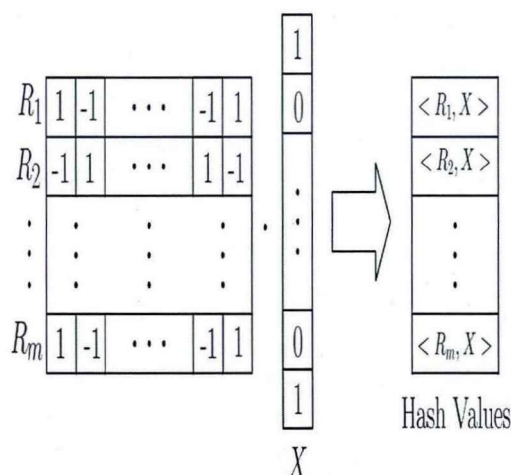


图 2.6: RAK-EEC 编码方案[6]

$[1, 2, \dots, m]$ 和 $S'_i, i = [1, 2, \dots, m]$ 就可以估算出传输过程中出错的比特数目。用于估错的理论如下：通过收到的hash函数值与计算得到的hash函数的值的差的平方的期望等于数据包中的错误数量；如果取 m 组hash值，重复上述过程，其平均值将更加接近于真实的值。形式化过程表示为：设原始数据包为 D ，收到的数据包为 D' ，其数据包中的误码率为 p_0 ，共有 m 组向量，表示为 $i = [1, 2, \dots, m]$ ，于是有 $E[(\langle D, R_i \rangle - \langle D', R_i \rangle)^2] = E[np_0]$ 。

文献[6]证明了上述过程的正确性，同时得到使用该方法得到的对误码率 p_0 的估算值 $\hat{p}_0$ 的精度可以达到文献[5]的估算精度。仿真和实验的数据显示，RAK-EEC相比于GS-EEC能够获得更加稳定的估错精度。随着误码率的变化，实验表明文献[5]中的估错的均方误(mean square error)变化更加平稳，准确度的衰减比GS-EEC更慢，其估算的偏差也更加小。

2.2.4 EToW-Sketch编码解码方案

在文献[7]，Hua等人中给出了一种更为简单有效的估错码编码方案。作者从通信复杂度角度建立了估错码的模型，分析估错码所需要的冗余的下界。同时给出了另一种基于hash函数的编码方案，在该文献中，这种编码方案被称为tug-of-war (EToW)方法。该文献中的实验和理论分析显示，EToW该编码方案冗余更少，估错的性能更为优良。

在文献[7]中，作者从通信复杂度的角度出发，给出了估错码需要的最低的冗余量。如果发送的数据记为 D ，收到的数据为 D' ，估错码是被看作求取两者之间最小汉明距的函数 $f(D, D') = \|D - D'\|_0$ 的值。经过理论分析，得到估错码需要添加的冗余的下界为 $\Omega(\log n)$ ，其中 n 为数据包 D 的长度。同时，该文献

证明了GS-EEC添加的冗余为 $O(\log n)$ ，符合通信复杂度的下界，因此GS-EEC的冗余量接近于最优。尽管如此，GS-EEC编码的冗余度仍然有降低的空间，例如可以通过减少 $O(\log n)$ 的系数的方法。文献中给出的方法EToW的编码方法的冗余度在保持下界的基础上，冗余度相当于GS-EEC的30%，同时能够取得更好的估算效果。

文献中[7]首先指出文献[34]中提出的一种基于估算2维的欧氏空间距离的方法ToW用以解决估算问题，该方法简单描述如下：

设 $\vec{s}_j \in \{-1, 1\}^n$ 为预定义的随机向量，其中 $1 \leq j \leq c$ ，发送端发送的数据为 $\vec{D}$ ，接收端的数据包表示为 $\vec{D}'$ 在发送端，计算出数据 $\vec{D}$ 的hash值 $Z = (z_1, z_2, \dots, z_c)$ ，其中 $z_j = \frac{\vec{D} \cdot \vec{s}_j}{2}$ 。在接收端，由于用于计算数据包hash值的 c 个向量 $\vec{s}$ 已知，则可以收到的数据包 $\vec{D}'$ 的hash值与收到的hash值之间的不同 $X_j = (z_j - \frac{\vec{D}' \cdot \vec{s}_j}{2})^2$ ，则误码率的估算值可以表示为 $\hat{p} = \frac{\sum_{i=1}^c X_i}{n}$ 。在基于ToW sketch的方法与RAK-EEC非常接近，但是估算公式不完全相同。事实上，如果将ToW sketch的结果做简单的变换，得到 $\hat{p} = \frac{\sum_{i=1}^c X_i}{n} = \frac{\sum_{j=1}^c (\frac{\vec{D} \cdot \vec{s}_j}{2} - \frac{\vec{D}' \cdot \vec{s}_j}{2})^2}{n}$ ，将RAK-EEC的估算结果做简单的变换，得到 $\hat{p} = \frac{\sum_{j=1}^m (\vec{D} \cdot \vec{s}_j - \vec{D}' \cdot \vec{s}_j)^2}{n}$ 。从上述两个公式的变形结果看，其估算的方法非常接近。

在文献[34]中，作者指出，这种方法存在的问题为，相比于GS-EEC，其添加的冗余信息接近于GS-EEC的冗余信息，同时Tow Sketch方法的容错能力很差：如果某个计算得到的数据包的轮廓信息在传递过程中出错。则可能影响整个估算结果的准确度。为了避免该种风险，在RAK-EEC中作者将计算的数据包的hash值和随机种子使用纠错码编码，用以保证一次传输正确，最终能够正确估算出误码率，但需要指出的是，即使纠错码，也有纠错能力的限制，一旦出现的错误超过了纠错码的范围，RAK-EEC仍然会因为hash值和随机种子的中包含错误使得估算的数据包误码率偏差太大。GS-EEC中使用两次随机编码，添加的冗余信息和数据位对数据包误码率贡献相同，因此避免了该种情况的发生。

在文献[34]中，作者给出了加强的Tow Sketch(EToW)方法，该方法的大体结构与ToW方法类似，不同地点在于：

- 根据不同的场景对估算精度的需求，在牺牲估算精度的情况下，降低添加的冗余信息。在ToW中，每个随机向量所计算出的数据包 $\vec{D}$ 的hash值 $z_j = \frac{\vec{D} \cdot \vec{s}_j}{2}$ 的取值范围为 $[-n, n]$ ，因此，要表示 z_j 需要的冗余信息为 $O(\log n)$ 比特。为了能够进一步降低添加的冗余信息，作者在估算精度和冗余大小之间做权衡，先从数据包中随机抽样长度为 l 的数据位，然后进行hash函

数的计算,通过这种方法,使得每一个hash值的长度为 $\log l$,且与 n 无关。这样就将冗余量从 $O(\log n)$ 降低到 $O(\log l)$ 。

- EToW方法中,作者根据用于估算的公式发现长度为 l 的hash值的高位所携带的信息没有低位所携带的信息大,因此可以使用更少的比特位只保存hash值的低位信息。在EToW中,增加了将多位hash值截断,只用很少的比特位保存低位的值,以此进一步降低冗余。
- 当某个hash值在传输过程中出错,使用这种出错的hash估算数据包的误码率会降低估算的精度,为了避免这种情况,在发送端,每个hash值需要额外的保护,EToW在发送端对每组hash值进行奇偶校验,在接收端,如果奇偶校验通过,则认为hash值在传输过程中没有出错,可以使用这些hash值估算错误误码率,若奇偶校验出错,则该hash值将不被使用。

文献中做了足量的理论分析和大量实验,证明当冗余量只是GS-EEC的30%时,其误码率的估算精度与GS-EEC相当,当冗余量与GS-EEC相同时,其估算的精度高于GS-EEC。

2.2.5 AMPS编码解码方案

在文献[9, 10]中引用了一种更为简单的估错码的编码方案: Amplified Sampling(AMPS),该估错码是估算数据包中的字节错误率的。在AMPS中,并不是准确判断出数据包中的字节错误的比例,而是将数据包分成若干段,通过计算每一段的错误字节数,并最终将计算得到的错误字节数放大,使得在最大后验概率下能够保证,放大的错误字节数是大于或者等于最大的错误字节数。这种放大的错误字节数可以使得系统重传足够的纠错码修正错误。该编码假设数据包中的字节错误率通常非常小,一般在 $[0, 0.2]$ 的范围内,通常为0.01。因此,如果采取从数据包中简单抽样的方法,通过判断抽样的数据中的错误数从而推断整个数据包的出错数会非常不准确。因此,AMPS没有采取简单采样的方式,而是将数据包分成若干段,在每段数据中使用校验的方法,一个校验比特校验多个字节。AMPS在接收端的计算过程如下:

- 在已知 X 个校验失败的情况下计算条件概率, $Pr(X = x|Y = y)$,其中 Y 表示真实的字节出错数。这里在数据包的传递过程中,文献中使用了交织技术,使得错误能够平均的分布在整数据包中,从而可以认为错误字节数服从二项分布。

- 利用最大后验概率公式计算 $P(Y = y|X = x)$ ，公式如下：

$$P(Y = y|X = x) = \frac{P(X = x|Y = y)P(Y = y)}{\sum_{y'=0}^U P(X = x|Y = y')P(Y = y')}$$

其中 $P(Y = y)$ 是先验概率，文献中通过引用思科的相关文献，确定出字节出错分布服从截断的幂次定律分布，并将次分布作为公式中的先验概率；

- 通过条件概率，在已知各分段数据中的错误字节数的条件下，计算出单个分段中出错数目的最大值。

从实验结果上可以看出，AMPS的在字节出错数目很小的情况下，估算的准确度高于GS-EEC，这得益于AMPS使用了两个非常重要的假设：

- 在真实环境中，字节出错率往往很小，该条假设被实验得以证明；
- 出错字节数目的分布服从截断的幂次定律分布（并且作者通过实验和理论确定参数）；

因此，相比于GS-EEC，在字节出错数目很小的情况下，利用已知的错误分布作为先验概率，其估错准确度高于GS-EEC。

2.2.6 Generalized-EEC的编码和解码

在文献[8]中，Nan等人从信息论的角度对GS-EEC和EToW进行分析发现：

- GS-EEC使用的估错算法的方差与Cramer-Rao界(一种对确定的参数估算产生的方差给出的下界)相差很大。于是作者构建出新的估错算法，并获得了非常高的估错精度，并证明这种估错算法接近于Cramer-Rao界，是近似最优的估错算法。实验结果同时给出了新给出对的算法在相同的冗余条件下，能够达到同EToW相同的性能，或者即使将冗余度降低到GS-EEC的30%，估错精度仍然与GS-EEC相当。
- 发现了一种称之为Generalized-EEC(gEEC)的估错码统一的框架，在该框架下，GS-EEC和EToW 两种估错码是在其参数不同的情况下的两种特例。
- 为gEEC寻找到了—种通用的解码算法即基于Jeffreys prior的最大似然估计算法（MLE）。该算法使得EToW的解码和估错性能更加优越。

本文在下面的内容中给出gEEC的编码方案:原始数据包用向量 $\vec{D}$ 表示,需要在数据包中增加的额外冗余信息假设为 m 个, $z_1, z_2, \dots, z_m$, 其中 z_m 计算过程如下:从 $\vec{D}$ 中随机有放回的选取 l_i 个数据比特记为 $\vec{D}_i$, 并与通过伪随机方法生成的校验向量 $\vec{s}_i$ 做异或运算(该过程与EToW类似, 与之稍有不同的是在EToW中, z_i 是衡量 $\vec{D}_i$ 中的1和0的数目的量)。

$$z_i = \sum_{j=1}^{l_i} (b_{i,j} \oplus \frac{1 + s_{i,j}}{2}) (\text{mod } 2^{k_i})$$

这里, 2^{k_i} 表示分配给 z_i 多少个比特保存计算结果。 $s_{i,j}$ 是从-1,1中通过伪随机的方法独立选择的值。

gEEC和EToW不同点在于在EToW要求所有的值必须相同, 即每次从原始数据中随机抽取的进行计算的比特数目相同;其次, 在EToW中, 所传输的每组校验信息 z_i 要通过投影的方法截断高位, 只保留低位的数据, 从而降低冗余量, 在上面计算 z_i 的过程中, 按照 2^{k_i} 取模和EToW相同。最后EToW中需要给所有的校验数据添加检错码, 从而保证使用的各组校验数据没有出错, 提高估错的精度, 而gEEC的解码算法具有容忍出错信息带来的不良影响。可以说, EToW是gEEC的在参数特殊化的特例。

上面的式子中 $1 + s_{i,j}$ 是将值域从-1, 1, 映射到0, 1上的变换, 我们将上面的式子做一个简单变形, 得到 $z_i = \vec{D}_i \cdot \vec{s}_i - \frac{1}{2} \vec{1} \cdot \vec{s}_i + \frac{l_i}{2}$, 如果我们对每个 z_i 只分配一个比特, 则要传输的对原始数据的校验值则变成了奇偶校验, gEEC也退化成为GS-EEC的形式, 这里 l_i 表示GS-EEC中每层校验位随机选择多少个数据位进行校验。

gEEC采用极大似然估计的方法估算出的误码率更为准确, 该方法同样适用于EToW和GS-EEC。由于推导复杂, 此处不再赘述。文献[8]中给出了一种估错的泛化的框架, 同时将两种主要的估错码表示为这种一般化的两个特例, 并且在充分进行信息论的推导后, 给出了更为有效的估算误码率的算法, 其性能被证明接近信息论的界, 近似最优。

2.3 数据包的错误修复协议

数据包的错误修复协议是指在网络传输过程中, 数据包出错后, 接收端通过重传多个副本或者重传纠错码等方式将出错的数据包修复为正确的数据包。重传的多个数据包副本并不一定是完整的数据包, 可以是原始数据包的某个切片; 重传的副本也不一定是完全正确的数据, 其在传输过程中可能出错。接收端将收到的各部分数据重新组合, 或纠正出错位, 最终恢复出完整的数据。

数据包的错误修复过程涉及自动重传请求机制。例如，如果原始的数据包在传输过程中出错，接收端通过校验发现错误，需要通知发送端重发部分数据或者重发副本。如果接收端正确接收数据包后，向发送端发送确认，使发送端更新缓存，准备发送下一个的数据包。

通常情况下，相比于有线传输场景，无线传输过程中数据包随时可能因受到干扰而造成数据包出错，丢包等情况也时有发生。因此错误修复技术在无线传输领域应用更加广泛。为了能够在不可靠的无线通信链路上发送和接收数据，针对不同的场景，有两种解决方案：

- 在实时性要求更高的场景，在重传的延时很大的场景和在单工链路的场景中，发送端把发送的数据使用前向纠错技术编码，在传输过程中，即使数据包出错，接收端也能够依靠前向纠错码将错误纠正，并最终解码成功；
- 在重传数据包需要的代价不大的情况下，发送端则在数据包中添加者检错码，使用自动重传请求应对信道的不可靠性，接收端在收到数据包后，对数据包的正确性做校验，如果无法通过校验，则丢弃该数据包，并请求重传该数据包，或者等待发送端超时重传。

可以看出，无论前文中采用的前向纠错技术或是自动重传请求，都有很大浪费。对于前向纠错而言，为了保证一次传输的成功，往往要添加大量的冗余，造成对带宽的浪费。如果使用自动重传请求机制，则当数据包有错误发生时，数据包中往往只有少量的错误，即误码率非常低，在这种情况下，简单丢弃含有错误的数据包相当于丢弃了大量正确的数据，从而造成浪费。文献[35]给出了一种极端的情况，该场景中，误码率往往低于0.001，当数据包的大小为12000比特时，完全正确传输一个数据包的概率大约只有0.00001，因此，如果要完全正确的传输一个数据包，则需要重传大约数十万次才能完成。事实上，如果采用出错数据包的拼接技术，即把若干个出错数据包拼接成没有出错的数据包，则只两次重传就能得到的正确数据包的概率增大到了0.99 上述事实说明了数据包的错误修复技术的重要性。

在无线网络的数据包错误修复技术中，有两个基本问题：

- 如何发现数据包出错。在有些场景中，不但需要通过编码计算出数据包的误码率，还需要获得数据包的出错位置的大体信息；
- 如何修复数据包中的错误。采用何种策略，将出错数据包修复为正确的数据包。

对于第一个问题,到目前为止,不同的工作给出不同的方案。从修复策略对设备的依赖程度上划分为两种类型:不使用硬件信息的错误修复技术和使用硬件信息的错误修复技术。这两种技术对应于信道编码解码过程中的硬判决和软判决方法。不使用硬件信息的策略是指,只通过修改上层的协议的方法,特别是数据链路层的协议,对物理层提交的包含有错误的数据帧做判断和纠正。在修复过程中,不使用物理层的其他信息。可以看出,这种修复过程只需要对设备的软件进行更新就可以达到目的。与解码的软判决类似,这种方法虽然耗费成本低廉,但计算量大,运算速度慢。另外一种方案是在修复物理层提交的数据帧中的错误时,使用物理层提供的其他信息。这种修复由于使用了额外的信息(例如使用收到的信号的强弱作为判定每个比特的信念),效果更佳,但却需要对设备的硬件进行升级,或者至少要刷新固件。

对于第二个问题,通常使用基于重传的机制修复出错的数据比特。一种可行的方案为在已知错误比特的粗略位置后,重传足量的纠错码纠正错误,这种方式对接收端而言,需要计算量较大,但是重传的冗余量较低,适合数据包中的误码率较小的情况。对于误码率大的信道,该种修复策略往往需要很大的计算量。另一种方案为直接重传数据包的出错的部分,当已知数据包的粗略出错位置,则发送端可以重传该部分,该种修复方法计算量较小,但是相比传输纠错码而言,传输量更大,这种策略更适合突发信道。在突发信道中,错误往往成块连续出现,通过重传出错的部分,只需要很小的计算量,就可以解决数据包错误修复。

2.3.1 不使用物理层信息的错误修复技术

在数据包的错误修复技术中,首要的问题是对错误的检测。在自动重传请求的策略中,往往是加入检错码。例如通过在发送端添加校验和,接收端重新计算校验和的方法检测是否有错误发生。文献[36]中作者认为,同一个源端以一个天线发送后,经过多路径的传输后,在接收端,由于信号之间的相互叠加干扰而造成出错。但是对于从同一个源端发送,经过不同天线和通信路径传输的数据,数据包出错的位置却是相互独立的。在该文献中利用系统MRD使用多天线,不同频率发送,接收端则使用多个网卡与之对应。由于不同的路径传输的信号出错位置相互独立,因此可以利用接收到的多个副本的组合将正确的数据修复出来。具体做法为:数据在发送之前切分成若干小分组,经过多路径传输后由于出错位置相互独立,则对于每个分组,收到的多个副本中至少存在一个分组是正确传输的。因此在接收端,对收到的每个分组的若干个副本做异或运算,判断分组是否相同。如果存在结果不为0的副本,则对于该分组而言,收

到的副本中存在错误。通过对出错分组的不同副本进行选择,拼凑出完整的数据包,然后对拼凑的数据包用循环冗余校验做验证,如果通过了验证,则认为该拼凑出的数据包正确,否则,选择出错分组的其他的副本组合方式。直到有某种组合通过验证。这样,通过多天线多路径的传输方法,实际上是使用了类似于重复码和循环冗余校验码的方法判断是否发生错误。通过对数据包分组判断出错的位置,并使用副本冗余修正出错位置。这种策略的优势在于不需要使用自动重传请求的方法修正出错位,降低了时延。但是当误码率很大时,数据包中的许多分组的副本都可能出错,则组合的情况变得非常复杂,计算量将非常巨大。

另外一种被称为二类混合自动重传请求[37]的策略也可以在不使用物理层信息条件下,完成数据包的错误修复任务。在这种混合自动重传请求中,使用纠错码对数据包编码,但是在传输过程中,首先传输添加了奇偶校验位的原始数据包。当接收端发现出现错误比特时,则向发送端请求重传请求,此时,发送端将发送足够的纠错码元以修正之前的错误。收到的数据包通过了校验,则认为没有出错,纠错码也不需要发送。许多不同的文章讨论并改进了这种混合自动重传请求的策略,如文献[3, 38–40]。

文献[3]基于上述思想实现了IEEE802.11上的数据包错误修复技术,该系统被称作ZipTx。该系统中,数据包被切分成更小的数据块,帮助定位错误的位置,ZipTx利用两回合编码纠正数据包中出现的错误。发送端首先发送没有添加奇偶校验信息的原始数据包,如果接收端通过冗余校验,发现其中收到的数据包中包含有错误,则将该数据包保存起来,接收端并不是对每一个包都要进行确认,而是对若干个包一起确认。经过若干个数据包后,接收端即向发送端发送确认信息,通知哪些数据包丢失,哪些数据包有错误。发送端通过这些确认信息决定重发丢失的数据包,或发送纠错码元。对于出错的数据包而言,接收端利用出错包收到的纠错码一起解码。第二回合中,接收端向发送端报告是否解码成功,如果解码没有成功,则发送端将继续发送更多的纠错码元,帮助接收端纠正错误,正确解码。如果上述尝试都失败了,则发送端重新发送原始数据包,重复新的两回合错误修复。ZipTx将每个数据包分成若干个分组,每个分组使用CRC做校验,使用RS编码做为纠错码。实验结果证明,通过ZipTx的修正技术,使用有错误的数据包,在发送速率很高的场景中,对通信的性能提升有限,但是在发送速率不高的场景中,性能提升非常大。

文献[4]中作者给出了另外一种在IEEE802.11中进行出错数据包的数据修复技术。该文献的核心在于认为真实场景中的数据包出错并不是随机的,而是突发,错误的位置是连续的。因此,将数据包分割成若干个分组后,如果发

生错误，则只需要重传连续的一些分组，或者分散的很少部分的分组就能达到对出错数据包的修正。为了判断数据包中的每个分组是否有错，分组中使用Fletcher-32一种循环冗余校验的变种的校验码做监督。文章中发送端和接收端的数据传输和重传请求，以及确认的基本原理与ZipTx类似。文献通过仿真和实验验证了这种方案的优势在于在不添加任何纠错码的情况下，能够降低修复的时延，并且这种方案可以被当前802.11所兼容。

2.3.2 使用物理层信息的错误修复技术

在文献[2]中，作者实现了另一种错误修复技术，这种技术被简称为PPR。与之前所介绍的错误修复策略不同，这种策略结合了两种非常重要的技术：

- 被称为软硬件层(SoftPHY)的硬件扩展接口可以向更高层提供物理层对比特位判决的依据。物理层提供的这种判决依据指，硬件层在解码时，每个接收到的符号或者码字有多接近解码后的符号或者码字。高层可以根据这个信息，在不需要发送端提供更多信息的情况下，通过对合法码字的对比判断，哪些数据更可能出错，哪些数据比特更可能正确；
- 在信号传输过程中，即使数据包的导言(preamble)部分出错，信号的结尾(postamble)部分也可以帮助修复数据。简单的讲，数据包的导言和结尾都是为了接收端进行同步操作而设定的，从而正确接收数据包而设置的。但通常情况下，由于噪音或者其他原因，导言部分同步会出错，但是结尾却能够正常接收。此时，使用回滚的方法，就可以确定出导言的位置，从而确定出数据包的起始位置和结束位置，收到的数据包也就可以正确解码了。

在使用上述技术修复出错数据包的同时，作者设计了对应的数据链路层的自动重传机制(PP-ARQ)：接收端很紧凑的选择数据包中的连续的位置，这些区域包含了所有的错误位置，然后请求发送端重传这些连续的区域。这样，接收端能够确定数据包中的所有比特都正确了，即完成了错误的修复。接收端使用动态规划的方法确定需要重传的连续区域，从而保证所耗费的通信开销最小。

文献[41]中作者认为在无线Mesh网络中，即使不存在任何一个节点能够完全正确接收整个数据包，但是，对于任何给定的比特位置，很可能被网络中的某个节点正确接收了。根据上述假设，一种被称为MIXIT的系统被设计出来。在该系统中，网络层及其以下的各层通过充分利用上述假设，提高了整个网络的吞吐率。通过使用软硬件层，物理层给出数据包中的每个比特判定的信念大

小。数据链路层不使用自动重传请求的方法对信念值低的比特位置做修正，而是将数据包及每个比特位置的信念值传递到网络层，网络层使用过滤器过滤掉低信念值的比特，再通过机会路由的方法，传输高信念值的比特到下一节点。

MIXIT的核心部件是使用一种新的网络编码，使得每一个工作的节点在很高的误码率情况下能够传输归属于不同数据包的符号的线性组合。这些符号是通过广播的方式发送的。这个核心部件需要解决两个问题：

- 控制副本的数量。每一个节点需要判定需要转发哪些符号，从而使得符号的传输副本不至于过大，MIXIT在其网络编码中的一种动态规划的方法里引入随机性解决这个问题；
- 修正错误。即使通过网络编码的方法，当目的端接收了足够的数据后，就可以解码出正确的数据包。但是，由于各个节点对比特正确与否的判定并不能做到100%，因此节点传输的具有高信念值的数据比特可能是出错比特，因此在目的端需要有纠错的策略。MIXIT使用了一种端到端的纠错部件解决网络编码中的出错问题，使得各个节点只需要负责传输高信念的数据比特。

文章的结论表明通过改变路由器的转发策略，即从只转发正确的数据包到只转发数据包的一部分正确的数据比特，从而获得了可靠的端到端传输，并且极大的提高了网络的吞吐量。

物理层的信息能够更好的帮助接收端修正出错的数据包，计算量小，性能也更加优良，但是由于需要对计算机硬件和物理层做修改，因此系统更新代价比较大，没有被当前主流的无线网络所取代。

尽管人们已经提出了多种差错控制技术，这些技术各有优缺点，或者是编码冗余量较大，或者是消息复杂度较大。研究人员仍然在探索低冗余、低开销的差错控制方法。EEC是目前冗余度最低的估错技术，但是它的纠错能力还没有被研究。本文将基于EEC，研究估错编码用于数据纠错的机制。

第三章 基于估错码的数据纠错算法

差错控制技术在无线通信中占有很重要的地位。在无线传输过程中，由于闪电等自然因素的影响和不同信道之间的干扰，使得无线传输变得非常不可靠，这导致接收端收到的数据中往往包含有错误。为了提供可信的数据传输，差错控制理论[42]被提了出来。该理论主要基于两种不同的技术，即前向纠错和自动重传请求。前向纠错技术的基本思想是发送端对原始信息添加大量纠错码，接收端收到经过编码的数据后，即使数据中包含有少部分错误，接收端也能够正确解码，而不需要请求重传。自动重传请求则是向原始信息中添加少量的检错码，如循环冗余校验码等，接收端发现收到的数据中包含有错误，则请求发送端重传之前的数据。目前，这两种技术都被大量使用。IEEE 802.11 使用自动重传请求机制和前向纠错机制来提高信道的可信传输[43]。

差错控制技术的目的在于，通过增加冗余或者使用重传，使接收端最终获得并使用正确的数据包。但是新近出现的许多应用可以使用出错的数据包[44–46]，这使得我们重新认识差错控制技术的目的。在这种情况下，文献[5]提出估错码的技术，旨在通过对数据传输过程中数据包误码率的有效估算，帮助更多应用使用出错的数据包。由于估错码只估算数据包中的误码率而不进行纠错，因此估错码能够以很低的低冗余量，和较低的计算复杂度的条件，取得比较高的估算精度。

纠错码和估错码是两种完全不同的编码技术，可以认为估错码是对纠错码的功能和其代价之间做权衡的结果。纠错码以很大的冗余量和计算开销为代价，试图保证在不可靠信道中的可靠传输，而估错码虽然能以很低的冗余量和解码复杂度有效估算数据包的误码率，但却不能够准确定位出错比特的位置，更不能纠正发生的错误。两种编码使用的场合完全不同，估错码使用在不可信链路中，被其编码的数据包中往往存在错误。一个非常有趣的问题是：估错码是否具备纠错能力？探究这个问题的意义在于：如果估错码能够纠正全部或部分出错的数据位，则在那些可以容忍出错数据包的应用中，可以减少数据包的误码率，从而减少不必要的重传，降低系统的时延，提高服务质量。虽然目前仍然缺乏估错码纠错能力的理论分析，但是既然估错码能够利用对原始数据随机的校验来估算误码率，则我们推测利用估错码的校验信息，至少可以修正部分出错的数据位。

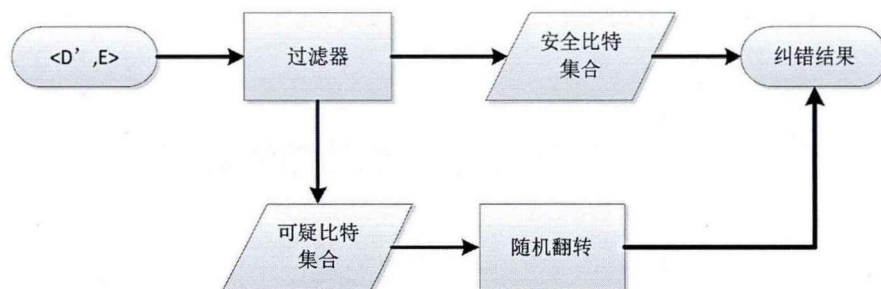


图 3.1: 纠错策略的流程

在本章中，首先探究估错码纠错的概率。研究显示估错码无法修正所有的错误，但却能够有效降低出错的比特数，特别当误码率很小的情况下，本文给出的策略能够修复大多数的出错比特。本文提出的错误修正策略是基于以下事实：在估错码的编码阶段，任何一个数据位都会被多个校验位校验。基于不同层的校验位的校验结果，本文引入过滤器算法将不太可能出错的数据位分离出来，以很大概率得到包含绝大多数出错数据位的集合——可疑比特集合。我们使用一种随机翻转算法尽可能的修正可疑比特集合中的错误。随机翻转算法通过随机地翻转可疑比特集合中的可疑数据位，生成一些可行的解，并输出能够最小化校验失败数目的解。理论分析显示我们的策略能在很大概率上使假阳性保持在很小的范围之内，同时能够修正绝大多数出错的数据位。

在本章中，我们使用真实的WiFi 访问日志中的数据包对提出的策略做了评估。本章中提出的基于估错码的纠错技术拥有广泛的应用。该技术作为对估错码的补充，可提升通信的质量，特别适合错误容忍的应用中。例如，实时的流媒体的应用可容忍错误的发生，使用我们提出的纠错策略能在不需要重传的前提下提升性能。该策略同时能够对新的传输技术提供帮助，例如基于误码率感知的数据包调度和重传技术在误码率超过某一阈值时选择重传，而本章提出的策略能够降低数据包的误码率，从而降低重传的概率。

3.1 基于估错码的错误修复

尽管本章的工作是基于文献[5]中提出的GS-EEC 编码，但本章中提出的算法框架，同样可以在RAK-EEC 以及EToW-EEC 和gEEC 上使用，因为这些估错码原则上都是基于奇偶校验或者hash 函数编码的。但是对于AMPS 而言，由于其估错是为了能够获得最小的足够大误码率，从而选用合适参数的纠错码，其估错算法和其他估错码不完全相关，因此，本算法框架不能完全应用到AMPS 上。

根据文献[5]对估错码的描述, 设置GS-EEC的相关参数如下: 数据包中有 n 比特数据位, 记为 $D = [d_1, d_2, \dots, d_n]$, 有 $l = \lfloor \log_2 n \rfloor$ 层校验位, 每层有 s 个校验比特, 因此数据包共有校验比特 $s \times l$ 位, 记为 $E = [e_1, e_2, \dots, e_{s \times l}]$ 。估错码的编码过程如下: 第 i ($1 \leq i \leq l$) 层的每个校验位从 n 比特数据中随机选择(有放回) $2^i - 1$ 位做奇偶校验。发送端发送的数据包中含有原始数据和校验信息记为 $\langle D, E \rangle$ 。接收端收到的数据包为 $\langle D', E \rangle$, 其中 $D' = [d'_1, d'_2, \dots, d'_n]$ 表示收到的数据包在传输过程中可能出错, 因此与原始的数据不一致。数据包的误码率用 p_0 表示。为了简化分析, 假设估错码校验位能够正确传输。出错的数据会使得接收端的部分校验失败。本章试图解决下面的问题: 在给定的条件下, 从接收到的数据中尽可能的修复出错的数据位。更详细的形式化描述该问题如下: 收到 n 比特数据 D' 和 $s \times l$ 比特校验位, 接收端对收到的数据进行校验, 得到校验结果矩阵, 定义如下:

$$C = \begin{pmatrix} c_{1,1} & c_{1,2} & \dots & c_{1,s \times l} \\ c_{2,1} & c_{2,2} & \dots & c_{2,s \times l} \\ \dots & \dots & \dots & \dots \\ c_{n,1} & c_{n,2} & \dots & c_{n,s \times l} \end{pmatrix} \quad (3.1)$$

其中, $c_{i,j}$ 表示校验位 e_j 对数据位 d'_i 的校验结果, 该值取三种可能的结果:

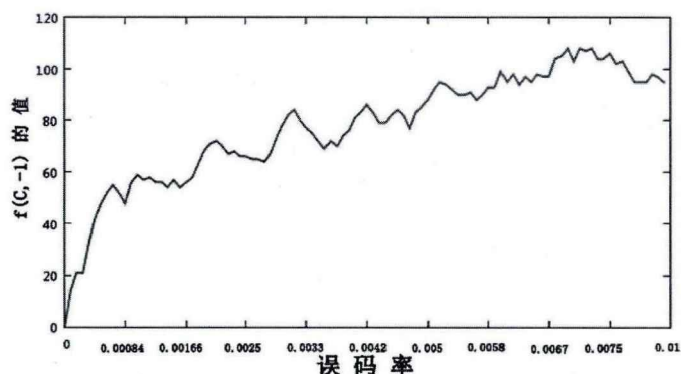
$$c_{i,j} = \begin{cases} 0 & \text{如果 } e_i \text{ 没有校验 } d'_i \\ 1 & \text{如果 } e_i \text{ 校验 } d'_i \text{ 并通过} \\ -1 & \text{如果 } e_i \text{ 校验 } d'_i \text{ 未通过} \end{cases} \quad (3.2)$$

定义函数 $f(C, x)$, 计算矩阵中的正项和负项的数目如下:

$$f(C, x) = \left| \sum_{\forall i \in [1, n], j \in [1, s \times l], c_{i,j} = x} c_{i,j} \right| \quad (3.3)$$

根据公式3.3, 当 $x = 1$ 时, 计算出矩阵 C 中所有取正数项的数目, 当 $x = -1$ 时, 计算出矩阵 C 中的所有取负数项的数目。若矩阵 C 中没有负数项, 说明所有校验都通过, 收到的数据正确, 即 $D' = D$; 如果矩阵中存在负数项, 说明数据中存在错误即 $D' \neq D$ 。本文以 $f(C, -1)$ 为目标函数, 提出一种纠错策略, 通过最小化目标函数 $f(C, -1)$ 尽可能多的纠正数据中的错误。

图3.2给出了在不同误码率条件下, 函数 $f(C, -1)$ 的值, 在计算这些值时, 我们使用随机生成数据包, 并随机产生固定数目的错误, 计算 $f(C, -1)$ 的值。

图 3.2: 误码率与优化目标函数 $f(C, -1)$ 的关系

其中的参数按照GS-EEC描述的那样设置。从图中可以看出, 函数 $f(C, -1)$ 随着误码率的不同, 且当 $p_0 \in [0, 0.01]$ 时, $f(C, -1)$ 随误码率 p_0 呈现递增的趋势。需要指出的是, 虽然在递增的趋势下, 在某些小区域内出现递减的情况, 如 $p_0 \in [0.0033, 0.0042]$ 时, 但在大量统计方面, 递增趋势仍然成立。因此, 使用 $f(C, -1)$ 作为优化目标, 寻找并纠正错误具有一定的合理性。

3.2 错误修复技术

与利用物理层信息纠错不同, 估错码没有物理层的任何信息, 只能利用估错码中有限的校验信息尝试最大可能纠正错误。根据这种情况, 我们提出一种出错数据包的修复技术, 旨在充分利用估错码的低冗余度, 并使用很低的计算复杂度, 尽可能纠正出错比特。图3.1中给出了完整策略的流程图。给定接收的数据和估错码校验信息, 本章首先引入过滤器算法, 通过利用两种规则, 对每个数据比特做快速判断, 将数据比特位分为两种类型: 安全数据位是被判定为传输中没有出错的数据位, 可疑数据位是被判定为传输中可能出错的数据位。可疑数据位组成的集合称为可疑集合。经过过滤后, 需要修复的出错数据位只可能出现在可疑数据位集合中, 因此问题解的搜索空间变小了。

事实上, 过滤器可以理解作为一种分类器, 将数据包中的数据比特按照指标分为不同的类型。其性能在很大程度上影响本修复策略的性能, 如果经过过滤器划分之后, 所有的出错比特位都被划分到可疑数据集合中, 则意味着这些比特位置将后续算法能够处理这些出错位置, 如果某些出错位置没有被过滤器划分到可疑数据比特集合中, 则这些数据比特将无法在后续算法中被纠正, 这种情况下, 整个纠错策略都只能试图尽可能减少错误数据包中的错误, 而无法修复所有的错误。上述对过滤器性能的描述可以用出错比特的假阳性的比例衡

量。过滤器另一个非常重要的性能衡量指标为可疑比特集合的大小。容易看出,可疑比特集合越小,后续的算法的复杂度越小(本章后面的内容将给出二者之间的关系),纠错策略整体的速度越快,反之亦然。两个极端的例子为,如果过滤器能够精确地将出错比特划分到可疑比特集合,则后续的算法一定能够纠正这些错误,并且耗费的时间将最低;如果过滤器将数据包中所有的数据比特都划分到可疑数据标题集合中,则后续的纠错算法将耗费极大的代价修复出错比特。可以看出,过滤器的这两个性能指标在一定程度上有冲突的地方。本章将在稍后的内容中展示给出的过滤器的算法的这两个性能。

通过过滤器后,本章给出的第二步纠错算法是一种随机算法,该随机算法在一轮的随机过程中,能够纠正的错误非常有限,甚至无法纠正错误,但是,当该算法重复很多轮时,纠错的性能就展示了出来。这正是随机算法的特点所在。本章给出的随机算法称为随机翻转算法。该算法通过随机删除可疑集合中的数据比特,翻转集合中剩下的比特,并使用评分函数评估翻转的有效性。该过程将重复若干轮,直到输出最好的翻转结果。该修复策略的细节在后续章节中给出。

3.3 过滤器算法

修复策略的第一步是使用过滤器算法分割数据包成为两个集合,安全数据比特集合和可疑数据比特集合。其目的是将正确的数据比特排除,降低最终解的搜索空间。过滤器算法是基于以下规则运行的。

规则1. 低层校验规则: 如果某一数据比特被低层的校验位校验,并且该数据比特通过了校验,则认为该数据比特在传输过程中没有出错,是安全的数据比特。

更形式化的描述为: 某一比特 d_i 当 $\exists j, 1 \leq j \leq \alpha s, c_{i,j} = 1$ 时, 该数据比特是安全数据比特, 其中 α 是正整数, 表示估错码校验位所在的层。该规则指出如果某个数据比特能够通过校验, 且该校验处在低于某一阈值的层内, 则该比特正确的概率很大。详细的分析如下:

根据估错码编码的规则, 每个第 i 层的校验位将随机选取 $g = 2^i - 1$ 位数据比特计算校验值, 由于每一个数据位出错的概率为 p_0 , g 比特数据位出错的数目服从二项分布 $B(g, p_0)$ 。设 Err 表示 g 比特数据中出错比特的数目, 则

$$Pr\{Err = 0\} = (1 - p_0)^g$$

$$Pr\{Err = 1\} = gp_0(1 - p_0)^{g-1}$$

因此我们得到:

$$Pr\{Err > 1\} = 1 - (1 - p_0)^g - gp_0(1 - p_0)^{g-1} \quad (3.4)$$

显然, 当 p_0 减小时, 公式3.4增大。令 $p_0 = \frac{1}{g}$, 则公式3.4可以表述为:

$$Pr\{Err > 1\} = 1 - (1 - \frac{1}{g})^g - (1 - \frac{1}{g})^{g-1} \quad (3.5)$$

容易证明, 公式3.5随着 g 的增大单调递减。当 $g = 31$ 时, $Pr\{Err > 1\} \approx 0.26$ 。这意味着, 当 $g \leq 31$, 并且 $p_0 \leq \frac{1}{g} = 0.032$, 则 g 比特数据位中出错数大于1的概率小于0.26, 当 p_0 更小时, 上述概率也将变得更小。

若某位校验位通过校验, 则说明所校验的 g 位数据位中包含了偶数个错误。根据上述分析, 当 p_0 和 g 足够小 ($g < 31$ 且 $p_0 \leq \frac{1}{g}$), 则该校验位校验的所有数据位中没有错误的概率等于 $1 - Pr\{Err > 1\}$ 。因此当某个低层的校验位通过了校验, 则所校验的数据位全部正确的概率将非常大。

需要说明的是第 i 层的每个校验位校验 $g = 2^i - 1$ 位数据比特, $g \leq 31$ 则得到 $i < 5$, 因此, 本文的后续章节中将阈值 $\alpha = 5$ 。

规则2. 高层校验规则: 如果某一数据比特没有被低层的校验位校验, 当该数据比特连续通过了高层校验位的校验, 则认为该数据比特在传输过程中没有出错, 是安全的数据比特。

更形式化的表述如下: 设某一比特位为 d_i , 如果 $\exists j$,

$$\sum_j^{j+\beta} c_{i,j} = \beta \quad (3.6)$$

则该数据位是安全数据位。其中 β 是正整数, 表示连续通过校验次数的阈值。

该规则表明即使数据比特位没有被低层估错码校验, 但却连续通过足够数目的高层估错码的校验, 该数据位也能被判定为安全的数据位。解释如下:

如上面所诉, 当误码率为 p_0 , g 位数据比特中出错的数据比特的数目服从二项分布 $B(g, p_0)$ 。我们定义函数 $\phi(g, p_0)$ 计算二项分布 $B(g, p_0)$ 中的奇数项的和如下:

$$\phi(g, p_0) = \sum_{\substack{Vi \in [1, g] \\ i \text{ is odd}}} \binom{g}{i} p_0^i (1 - p_0)^{g-i} \quad (3.7)$$

通过推导, 我们可以证明:

$$\phi(g, p_0) = \frac{1}{2}(1 - (1 - 2p_0)^g) \quad (3.8)$$

当 g 位数据比特被一个估错码校验时, 其中的一个出错比特 d'_i 能够通过校验, 当且仅当在剩下的 $g - 1$ 位数据比特中, 存在奇数个错误, 这种情况的概率为 $\phi(g - 1, p_0)$ 。当 $p_0 \in (0, 0.5)$, $g \geq 2$ 时, $\phi(g - 1, p_0) < \frac{1}{2}$ 成立。因此 d'_i 能够连续通过 β 个估错码元校验的概率可以被界定在下面的公式中:

$$Pr(d'_i \text{ passes } \beta) < \left(\frac{1}{2}\right)^\beta \quad (3.9)$$

可以根据不同的应用需求, 选取不同的 β 的值。 β 越大, 就越难通过过滤器, 能通过过滤器的数据比特就越少, 当 $\beta = 5$ 时, 错误的比特位能够通过过滤器的该概率将小于0.04。

上述两条规则将数据比特划分到两种集合中: 安全比特集合和可疑比特集合。我们使用假阳性作为衡量过滤器性能的指标, 假阳性定义为出错比特被划分到安全比特集合的概率。下面的理论给出过滤器算法的性能的下界。

定理 3.1. 本文中提出的过滤器算法的假阳性出现的概率将小于 $\max\{1 - \frac{2}{e}, (\frac{1}{2})^\beta\}$

证明. 在规则1中, 出错比特通过低层估错码元的概率的界被下面的公式所给定:

$$Pr\{Err > 1\} = 1 - \left(1 - \frac{1}{g}\right)^g - \left(1 - \frac{1}{g}\right)^{g-1} < 1 - 2\left(1 - \frac{1}{g}\right)^g$$

根据文献[47]中

$$\frac{1}{e}\left(1 - \frac{1}{g}\right) \leq \left(1 - \frac{1}{g}\right)^g \leq \frac{1}{e}$$

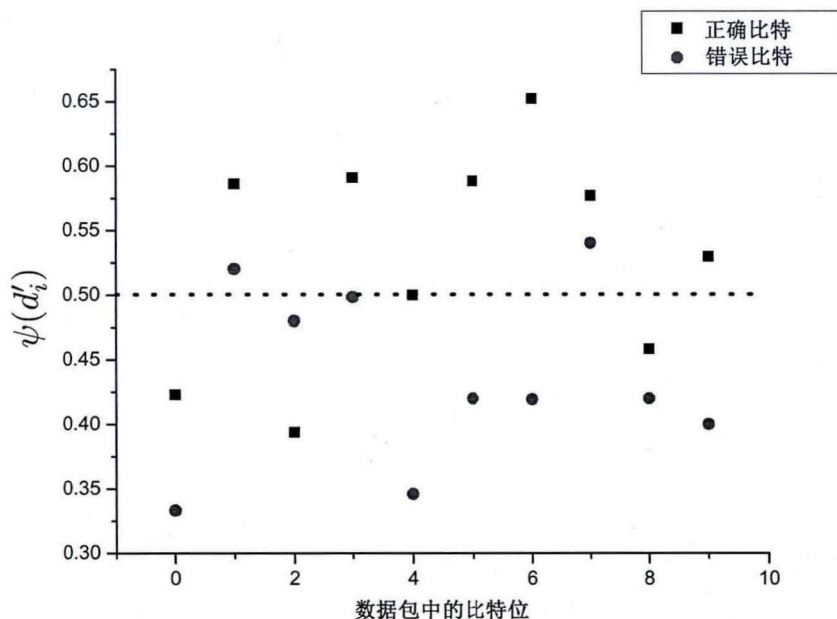
得到:

$$Pr\{Err > 1\} < 1 - \frac{2}{e}\left(1 - \frac{1}{g}\right)1 - \frac{2}{e}$$

由公式3.9得到, 在规则2, 出错比特能够连续通过 β 个高层估错码元校验的概率小于 $(\frac{1}{2})^\beta$ 。因此, 本文提出的过滤器算法的假阳性的概率低于 $\max\{1 - \frac{2}{e}, (\frac{1}{2})^\beta\}$ 。

□

综上所述, 当算法的参数选取合适时, 应用本文给出的过滤器算法, 得到两个集合, 其中的可疑数据位集合中包含了大多数的出错比特。


 图 3.3: 随机选取的比特的 $\psi(d'_i)$ 函数的值

3.4 随机翻转的错误修复算法

设得到的可疑集合为 S ，其中的元素数为 m ，本小节将提出一种随机算法修复在集合 S 中包含的出错比特。利用文献[5]中的方法，估算数据包的误码率，用 $\hat{p}$ 表示，并由此计算得到集合 S 中的出错比特的数目 $\hat{r} = \hat{p}n$ 。由于无法计算出 r 的精确值，如果穷举所有可能的解，则有 2^m 种组合需要证明，这种复杂度不可接受，因此我们提出了一种随机算法来修复在集合中的出错比特，理论证明这种算法的复杂度为 $O(\log(m^r \log(m)))$ 。

本文提出的随机翻转算法基于下面这个观察结果：若某个数据比特位正确，则更容易通过接收端的校验。

设 $\omega_1(d'_i)$ 和 $\omega_2(d'_i)$ 分别表示数据比特 d'_i 通过和没有通过的校验的数目，则

$$\omega_1(d'_i) = \sum_{\forall j \in [1, s \times l], c_{i,j}=1} c_{i,j};$$

$$\omega_2(d'_i) = \sum_{\forall j \in [1, s \times l], c_{i,j}=-1} |c_{i,j}|.$$

定义 $\psi(d'_i)$ 为数据比特 d'_i 通过校验的比例：

$$\psi(d'_i) = \frac{\omega_1(d'_i)}{\omega_1(d'_i) + \omega_2(d'_i)} \quad (3.10)$$

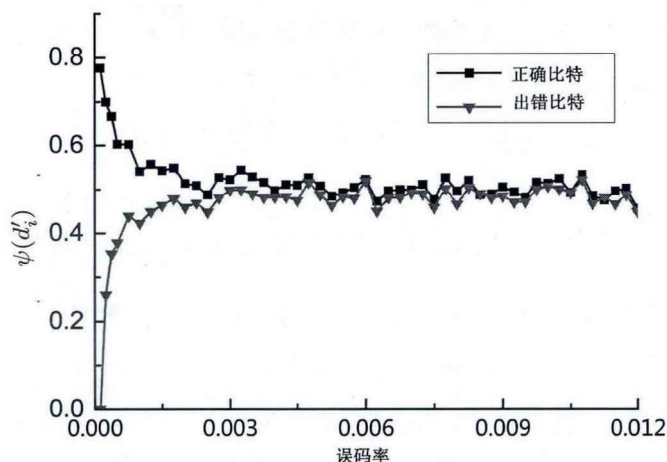


图 3.4: 出错比特和正确比特 $\psi(d'_i)$ 函数随误码率的分布

在理想情况下，正确比特的 $\psi(d'_i)$ 应该大于0.5 而出错的比特的 $\psi(d'_i)$ 值应该小于0.5。图3.3展示了在误码率为某个值的情况下，任取的十个正确的比特和十个错误的比特的 $\psi(d'_i)$ 的值。通过对比可以看出，正确的数据比特总体而言， $\psi(d'_i)$ 的值大于0.5但是有三个正确数据比特的值小于0.5，对于出错比特而言，整体上都 $\psi(d'_i)$ 的值整体上都小于0.5，但是，也存在两个出错数据比特大于0.5的情况。

图3.4展示了不同的误码率的条件下， $\psi(d'_i)$ 函数的平均值。图中，当误码率小于0.003 时，正确比特的 $\psi(d'_i)$ 函数的值远大于0.5 而出错比特的 $\psi(d'_i)$ 函数的值小于0.5。随着误码率的增大，我们可以看到正确比特和出错比特的 $\psi(d'_i)$ 函数的值都趋近与于0.5，但是对于大多数出错比特， $\psi(d'_i)$ 小于0.5仍然成立。

基于上述观察结果，我们提出一种随机算法——随机翻转算法用以修复出错的数据比特。该算法的具体描述如下：

算法 3.1. (1)对集合 S 中的每一个数据比特 d'_i ，计算 $\psi(d'_i)$ ；

(2)以 $\psi(d'_i)$ 作为删除 d'_i ($i = 1, 2, \dots, m$)的概率大小，随机从集合 S 中删除数据比特 d'_i ；

(3)重复上述步骤直到 $|S| \leq 2\hat{r}$ ；

(4)穷举集合中元素所有的可能组合，针对每一种组合，翻转这些组合中的比特位，计算 $f(C, -1)$ ，输出使得 $f(C, -1)$ 最小的那个组合。

如果 $f(C, -1) = 0$ ，则意味着所有的数据比特都通过了校验位的校验，并修复了所有的错误。如果 $f(C, -1) \neq 0$ ，上述算法使得 $f(C, -1) = 0$ 最小，即

减少了输出结果中错误。由后面章节的实验可以看出,该算法在误码率较小的情况下,能够以很大的概率修复大多数的错误。

为了能够加强该算法的效果,可以将该算法重复 m^r 次,得到最优的输出结果。

定理 3.2. 本文提出的随机翻转算法能够以很大概率纠正所有的错误。

证明. 对于出错比特 d'_i , 根据 $\psi(d'_i)$ 的性质,得到经过步骤2后, d'_i 仍然在集合 S 中的概率为 $1 - \psi(d'_i) > \frac{1}{2}$ 。同理对于正确比特,从集合 S 中删除的概率大于 $\frac{1}{2}$,步骤2和步骤3将重复 $\log m$ 次。因此,错误比特经过 $\log m$ 次删除的过程后,仍然在集合 S 中的概率将大于 $(\frac{1}{2})^{\log m} = \frac{1}{m}$,而所有的 r 比特错误都在集合 S 中的概率将大于 $\frac{1}{m^r}$ 。

在集合 S 中的 r 比特错误将在步骤4中被穷举,因此一定能够得到纠正。因此, r 比特错误不能被完全纠正的概率将小于 $1 - \frac{1}{m^r}$ 。

经过对步骤1到步骤4重复 m^r 次,则 r 比特错误不能被完全纠正的概率将小于

$$(1 - \frac{1}{m^r})^{m^r} < \frac{1}{e} \quad (3.11)$$

因此,所有的出错比特能够以很大的概率纠正。□

定理 3.3. 随机翻转算法的复杂度为 $O(m^{r+1} \log m)$

证明. 步骤1中的复杂度为 m^{sl} ,由于估错码的冗余量小,因此 sl 的值非常小,可视为常量。步骤2中的复杂度为 $O(m)$,步骤3中进行删除次数不超过 $\log m$ 次。步骤4中所需的计算复杂度不超过 2^{2^r} 次。

随机翻转算法重复 m^r ,当误码率很小时,集合 S 中出错比特数 r 及其估计值 $\hat{r}$ 都非常小,可以视为常数,因此,总的算法复杂度可以表述为:

$$(m^{sl} + O(m) \log m + 2^{2^{\hat{r}}} * m^r) \sim O(m^{r+1} \log m)$$

□

综上所述,当误码率 p_0 很小时, r 可以被视为常数,本中提出的随机翻转算法属于多项式复杂度,将远远低于 2^m 。

3.5 性能评估

在上一小节中,本文所提出了利用估错码纠错的核心策略。在本小节中,将对提出的纠错策略做理论性能方面的验证,我们实现了上述算法,并用真实的WiFi访问日志中的数据集做测试,测试结果显示本文提出的算法,能够在很大程度上纠正数据包中的错误,降低数据包中的出错比特数目。

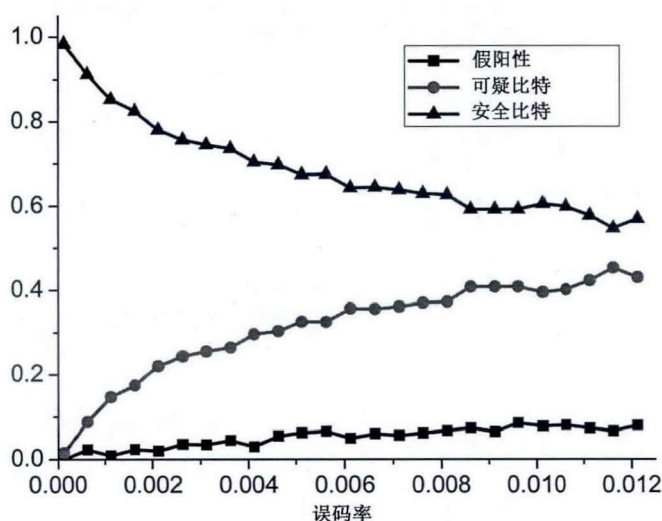


图 3.5: 过滤器算法在不同误码率下的性能

3.5.1 WiFi访问日志

我们使用真实WiFi访问日志测试文中提出的算法性能。该实验数据是从达特茅斯学院无线数据收集社区网站crawdad下载而来，对该社区的介绍可参考文献[48]。该社区致力于帮助科研人员分析来自于不同地方的各种数据，从而更好的揭示无线网络的某些特性。每年都会增加许多各种不同的数据。该网站上的许多数据被用于广泛的研究，其成果发表在诸如ACM SIGCOMM等顶级的会议和期刊上。本文所使用的数据源于密歇根州立大学的无线和视频实验室收集和公布的真实WiFi访问日志。这些记录的收集过程如下：

如图A.1在典型的办公室环境中，设置6个接收器，用于同时发送端接收数据。其中的三个接收器被放置一个房间中，该房间与发送器隔着一个走廊。另外三个接收器被放置在距离100米远的其他房间，这样，这三个接收器就将处在网络的边缘。发送端通过多播向这六个接收端发送数据，同时发送端禁用MAC层的重传机制，并将物理层的自动速率选择功能禁用，每个AP使用固定速率进行传送，每组实验中使用不同的数据传输速率。使用802.11b协议进行传输。这些记录大约有超过9000000个数据包，每个数据包中包含1000字节的数据，每个字节传输十六进制的“AA”即二进制的“10101010”，实际的比特错误可以通过计算真实收到的字节与“AA”异或后得到。其中有超过10%的数据包包含有错误数据位。每一个数据包大小为 $n = 8000$ 位数据比特，在本文的实验中，设置估错码共 $l = \lfloor \log n \rfloor$ 层，每层使用 $s = 32$ 个估错码校验位。

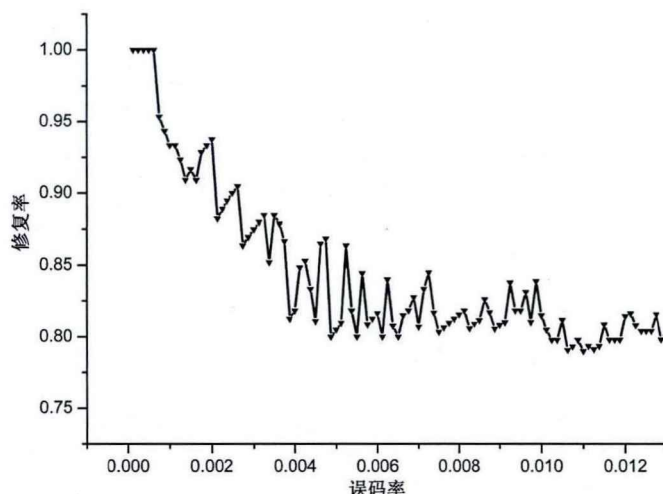


图 3.6: 随机翻转算法在不同误码率下的性能

3.5.2 过滤器有效性评估

图3.5中显示了在使用过滤器算法后，安全数据位和可疑数据位和假阳性所占的比重。由图可知，当误码率很小时，过滤器算法能够得到只包含很少可疑数据位的集合。当误码率小于0.003时，可疑集合大小将不足原始数据包的25%，这意味着，解的搜索空间可以降低到原始数据包的 $\frac{1}{4}$ 。当误码率增大时，安全比特和可疑比特集合的大小将趋近于原始数据大小的50%，尽管如此，过滤器仍然能够排除50%的数据，使计算复杂度降低。总之，过滤器算法能够在误码率小的情况下，极大的降低问题的求解规模。同时，过滤器假阳性的错误非常小。在误码率低于0.003时，假阳性率不足5%，即使误码率增大，假阳性的率也不足15%。

3.5.3 随机翻转算法的性能

本文将被纠正的数据比特数占出错的数据比特的比重定义为数据修复率。图3.6显示在不同的误码率下随机翻转算法的修复率。当误码率很小时，数据的修复率高于90%，当误码率增加时，修复率下降的很快，降低到不足60%，其原因有两方面：第一，当误码率很大时，过滤器产生的可疑集合变大，导致随机翻转修复错误变得更加困难。其次，当误码率增大时，函数趋近于0.5，这使得在随机翻转中区分出错数据位变得非常困难。因此错误修复率随着误码率的增大降低了。

图3.7比较了使用随机翻转算法前后，数据包的误码率的累积分布图。该图展示了原始记录中，超过90%的误码率为0，经过随机翻转算法，误码率为0的

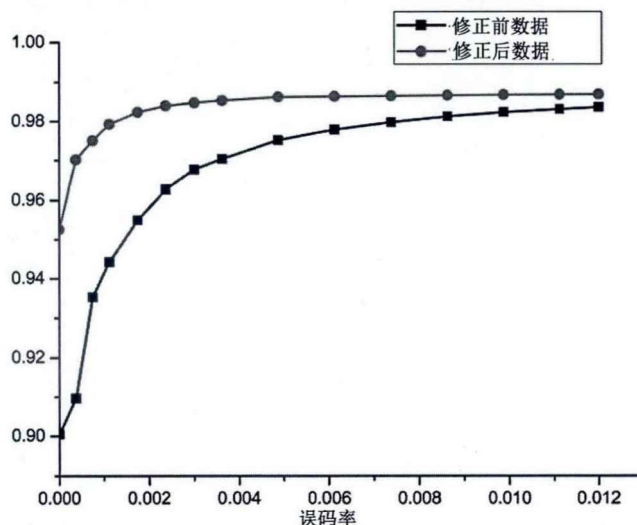


图 3.7: 随机翻转算法前后访问日志中误码率的累计分布图

数据包增加到94.5%。这表明有4.5%的数据包的错误被纠正，成为完全正确的数据包，占出错数据包数量的近45%的数据包被完全修复。经过修复后，从累积分布图中可知，即使误码率比较大，修复策略也有效降低了误码率。

3.6 本章小结

本章研究了使用估错码修复数据的可能性。本文将数据包中误码率大小与校验位校验失败率联系起来，将错误比特的修复问题形式化为一种优化问题：即更少的错误意味着数据包中更低的校验位校验失败率，通过选取并翻转一些比特的组合，使得校验位的校验失败率降到最低。这种修复策略首先引入一种过滤器算法，该算法利用两种基本的规则，将每个比特划分到不同的集合中，其中可疑比特集合包含有大多数出错比特。这样，要纠正出错的比特，只需要在可疑比特集合中进行，从而降低问题的求解空间。其次，使用一种被称为随机翻转的算法。该算法先根据比特的校验结果，选择删除足量的比特，再对剩下的比特做穷举，得出可疑最小化目标函数的解。通过不断重复上述过程，就能够以很大概率纠正所有的错误。

在本章中，我们对提出的过滤器算法的性能和随机翻转算法的性能做了理论分析，并对随机翻转方法的计算复杂度给出了分析的结果。同时，使用真实的WiFi访问日志做了模拟，结果显示过滤器算法和随机翻转算法的有效性，本章提出的基于估错码的纠错算法能够在误码率足够小的情况下以很大概率纠正所有的错误，在误码率较大的情况下，有效降低误码率。

第四章 EEC-PPR：基于估错码的数据包修复协议

本文在第三章中讨论了估错码(GS-EEC)的纠错能力，认为估错码在估算数据包误码率的同时，具有一定的纠错能力。文章给出了一种纠错策略，使得在误码率足够小时，该策略可以以极高的概率纠正出错的数据；在误码率较大的情况下，策略能有效减少数据包中的错误数量，从而提高数据包的正确率，提高容错应用的QoS。但是上述讨论仅局限在算法和理论工作，并没有应用在具体的场景中。在本章中，我们将第三章的纠错策略运用到差错控制协议中，设计出一套基于估错码的出错数据包修复协议：EEC-PPR。该协议与传统的出错包修复协议不同点在于，充分使用估错码技术估算数据包的误码率，同时使用估错码的纠错功能，在误码率足够小的情况下的高纠错能力自动纠正数据包中的错误，从而避免了重传，避免降低了时延。

4.1 概述

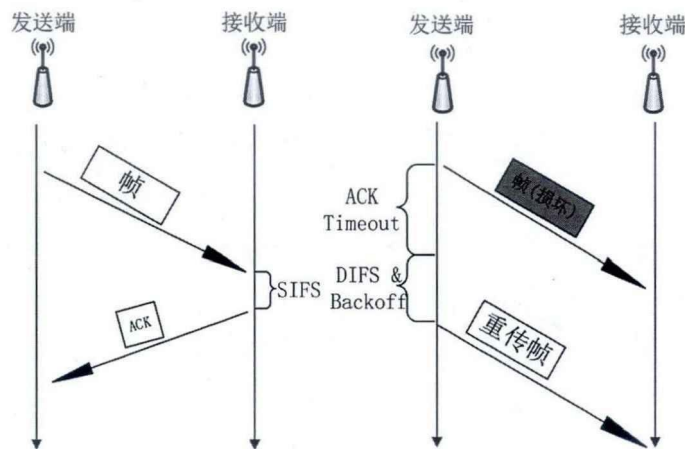


图 4.1: 当前802.11中的自动重传请求

图4.1中展示的是在IEEE 802.11 的数据链路层使用的自动重传请求协议。在该协议中，发送端在发送一帧数据后，将启动计时器等待来自接收端的确认信息ACK。接收端在经过了短帧间间隔（SIFS）后，即向发送端发送确认帧。如果发送端收到确认帧，则继续发送下一个数据帧。如果接收端收到的数据帧包含有错误，则接收端将不发送确认帧。当发送端的计时器超时后，发送端则认为之前发送的数据帧可能出现错误并重传该数据帧。在重传之前，发送端必

须首先监听直到信道空闲, 等待的时间长度为分布式帧间 (DIFS) 时长, 并随机退避一定的时间。可以看出在当前802.11的协议中, 如果接收端收到了数据帧, 该数据帧包含有错误, 则接收端并不通知发送端错误的的数据以及错误的类型等重要的反馈信息, 而是简单丢弃, 并等待发送端重传。对于发送端而言, 必须等到确认的计时器超时才能重传该帧。

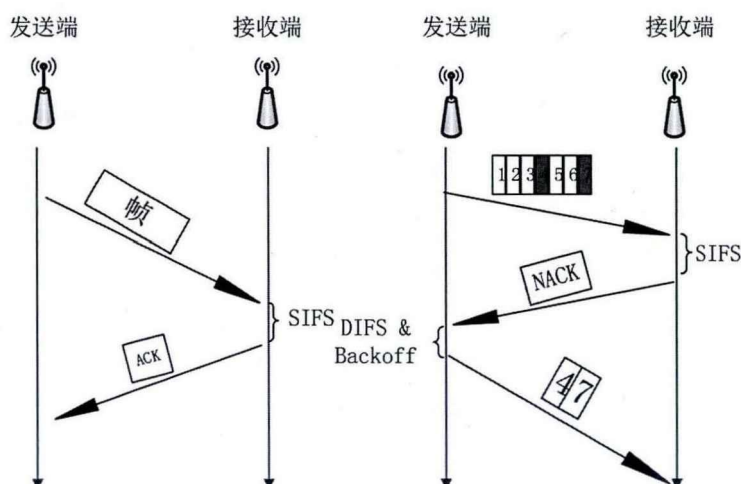


图 4.2: 基于EEC的出错包修复EEC-PPR

图4.2展示了本文对IEEE 802.11的修改, EEC-PPR中, 我们修改了数据链路层的自动重传请求协议。在该协议中, 当接收端收到了包含有错误的数数据帧, 由于在发送端发送的数据帧中将数据帧分为若干分组, 同时通过使用估错码编码, 接收端可以估算出数据的出错数量, 当误码率很大时, 本文提出的EEC-ARQ使得接收端得到错误位于哪一个分组中, 并向发送端发送错误确认 (NACK), 该错误确认帧中, 将指出哪几个分组出现错误, 发送端通过错误确认帧获得传输反馈, 向接收端重传出现错误的分组, 而不需要等待确认超时后才重传整个数据帧。这种设计的优势在于:

- 降低纠错所需时延。由于接收端可以更快的向发送端反馈收到的数据帧的情况, 因此, 当数据包中出现错误时, 发送端不需要等待超时才重传数据帧, 而是在接到反馈后即可重传。在信道不稳定白噪声大的情况下, 这种时延非常大, 而本文中提出的方案可以很大的降低这种时延。
- 降低纠错所需冗余。由于在传统的IEEE 802.11中, 一旦出现错误, 需要重传的整个数据帧, 这将带来很大的浪费, 在EEC-PPR中, 当误码率足够小的时候, 甚至不需要重传就能够纠正数据帧中的错误, 当误码率较大时, 需要重传的只有出错的分组。

在对估错码研究过程中,我们发现了很重要的两个事实,本文中所提出的EEC-PPR的设计,正是基于这两个事实:

- 估错码在估算误码率的同时具有一定的纠错能力:在误码率足够小的情况下的具有很好的纠错性能,能够接近100%的纠正所有的错误;当误码率较高的时候,通过运行纠错算法,能够有效的降低数据包中的误码率;
- 在WiFi的信道中错误发生的情况具有一定的统计规律:
 - 在所有的出错数据包中,数据包中误码率的分布遵守幂次定律分布;
 - 在出错的数据包中,错误的类型往往是突发类型错误,而非随机错误。

基于上述事实,我们设计出一种基于估错码的出错数据包的修复协议EEC-PPR,充分利用估错码的误码率估算和纠错能力,降低数据包重传次数和重传的数据量,减少时延,节省带宽。

4.2 WiFi信道的错误分布

4.2.1 数据包误码率符合幂律分布

在第三章的工作中,我们使用了一个真实的WiFi访问日志验证本文提出的基于估错码纠错算法的性能。在研究中发现,真实WiFi访问日志中的错误在数据包中的分布具有一定的规律。通过对整个WiFi访问日志的所有数据包的各种不同的统计,我们得到如下结果:数据包的误码率的分布符合幂次定律分布。图4.3和图4.4中,横坐标表示误码率的大小,纵坐标表示该误码率占错误数据包的数量比例。黑色的点表示从真实WiFi访问日志统计中得到的值,红色点线表示按照幂次定律拟合出的结果。我们使用Origin工具进行拟合,多轮迭代后,拟合收敛。图4.3给出了拟合的相关参数,得到的公式如下:

$$P(x) = 1.30904 \times 10^{-4} x^{-0.57738} \quad (4.1)$$

在经过拟合后得到的公式4.1中 x 表示数据包中的误码率,可以很清楚的看到,真实的WiFi访问日志的误码率分布非常类似幂次定律分布,我们进一步统计两个不同WiFi访问日志中关于误码率的信息,

统计误码率为前20%的数据包和误码率小于0.003的数据包,Origin工具导出的统计表格如下所示:

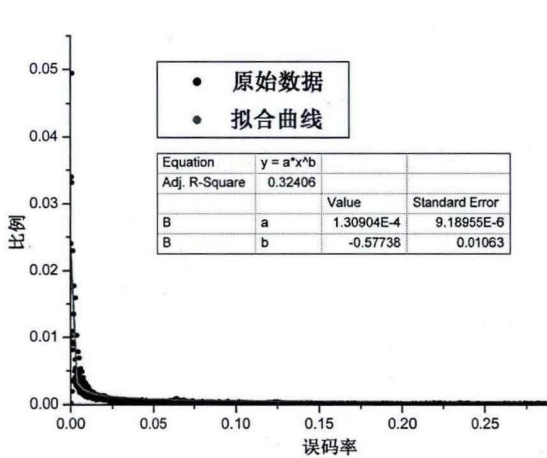


图 4.3: 数据包中误码率的分布

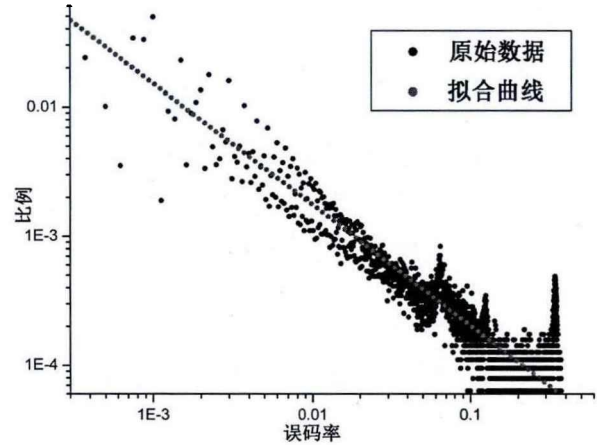


图 4.4: 数据包中误码率的log-log分布

表 4.1: 真实WiFi访问日志中出错数据包中的误码率统计表一

统计总数	平均值	标准方差	和	最小值	中值	最大值
2966	3.37154E-4	0.00162	1	1.57791E-5	9.46746E-5	0.04942
600	0.00125	0.00344	0.74968	1.26233E-4	4.89152E-4	0.04942
24	0.0123	0.0124	0.29518	1.26233E-4	0.00854	0.04942

在表格4.1中，统计总数指误码率从 $\frac{1}{8000}$ 到 $\frac{3}{8}$ ，共统计了2966种不同的误码率，（有些误码率的取值在数据集中没有出现）其中，误码率从 $\frac{1}{8000}$ 到 $\frac{3}{40}$ 大约以20%的统计量，占到了所有数据集出错数据包的74.9%，非常接近2-8分布。我们同时统计了误码率小于0.003的出错数据包所占的比例，结果显示，该种类型的出错数据包大约占有所有出错数据包的30%。在表格4.2中，共对误码率统计了2271个不同的取值，从 $\frac{1}{8000}$ 到 $\frac{3}{8}$ ，误码率从 $\frac{1}{8000}$ 到 $\frac{3}{40}$ 大约以20%的误码率跨度，占出错数据包的93%，对误码率小于0.003的出错数据包所占的比例，结果显示，该种类型的出错数据包大约占有所有出错数据包的51%。上述统计表的结果和图4.3与4.4的结果揭示了真实WiFi误码率的分布符合幂次定律分布。

图4.5和图4.6中分别给出了一个WiFi访问日志中误码率的概率分布和其累积分布（在概率分布图中，没有给出误码率为0时的值，因为该值非常大）。从

表 4.2: 真实WiFi访问日志中出错数据包中的误码率统计表二

统计总数	平均值	标准方差	和	最小值	中值	最大值
2271	4.40335E-4	0.00387	1	4.84147E-6	4.35732E-5	0.15753
600	0.00156	0.00741	0.93391	4.35732E-5	2.27549E-4	0.15753
24	0.02145	0.03079	0.51488	3.92159E-4	0.01285	0.15753

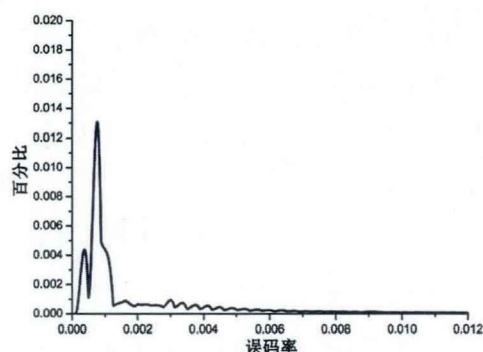


图 4.5: WiFi误码率的概率分布

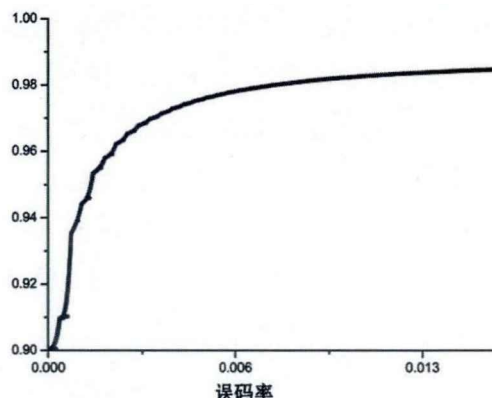


图 4.6: WiFi误码率的累积分布

累计分布图中可以看到，误码率为0的数据包占了整个访问日志的90%，这说明在典型的办公室环境中，802.11网络的信道相对稳定，出错率大约为10%左右。我们统计了所有的WiFi数据集的误码率，结果显示，出错率所在的范围为1.7%到30%。平均值为20%左右。

从该图和揭示的分布律中，可以得到两个非常重要的结论：

- 出错的WiFi数据包的误码率往往非常小；特别的，误码率小于0.003的数据包占据了出错数据包的近30%到50%。该现象在文献[2]已有相关的讨论。在有线网络中，重传完整的数据包非常有用，因为数据出错非常少见，且一旦出错，往往是交换机等的缓存溢出，造成整个数据包丢失。因此，在有线网络中，这种重传整个数据包的能够很好的解决常见的丢包问题。对于无线网络而言，需要发送端重新传输数据包的情况除了丢包，数据包因为干扰和噪声而导致出错是更为常见的问题。出错的数据包的绝大部分数据比特是正确的，由于出错出错的数据比特少，重传整个数据包意味着传输了额外的冗余信息，造成网络传输的极大浪费浪费。
- 平均情况下，出错的数据包占传输数据包总数的20%以上，经过统计，最糟糕的数据集中，出错的数据包占到30%以上。这意味着，相对于有线网络而言，出错出错的数据包的数量大大增加，但同时仍然有相当一部分数据包能够正确传输。这就使得通过增加高冗余的纠错码纠正出现的错误的做法不但浪费网络开销,同时大量计算造成计算资源的浪费。

4.2.2 WiFi信道中的错误类型

文献[4]中，作者给出的统计结果显示在真实的WiFi数据传输中大约有60%数据包在传输中出错，大约有5%的数据包在传输过程中因为同步等

问题丢失, 仅有大约35%的数据包能够完全正确的传输。根据我们对数据集中成功接收的数据包的统计(因同步失败导致接收端未能接收到的情况没有包含在内)收到的数据包的出错率, 在不同的实验中各有不同, 从1%到30%, 平均占20%左右。即收到的数据包中, 大约有20%的数据包中包含有错误。本文对数据错误的分布和错误比特的位置的相关性做了对应的统计, 得到这样的事实: 数据包中的错误比特的位置具有某种相关性, 错误并不是随机发生的。即WiFi在传输过程中的错误往往是突发错误, 而不是随机单比特错误。

众所周知, 在传输过程中因为信号受到电磁干扰, 出现的错误往往分为两种, 单一比特错误和突发错误。单比特错误是指数据在传输过程中, 只有“0”变成“1”或者“1”变成“0”这种错误, 错误和错误是随机发生, 且相互独立的。突发错误是指, 在连续区域, 同时有多个比特位发生跳变, 其出错的比特位置不一定连续, 但却集中在某一区域内。突发错误一般是受到持续的电磁脉冲干扰造成的。

我们首先统计了数据包中出错的比特之间的最大距离随着误码率的变化情况, 其次反设所有的错误是随机错误, 推导上文提到的变化情况, 通过对比这两种数据得出结论。

设数据包中有 t 个出错比特, 将这些比特出错的位置按照从小到大的顺序排列后所处的位置在数据包中用 $X_1, X_2, \dots, X_t$ 表示, 其中 $X_i \in [0, n]$ (这里 n 表示数据包的长度)如果这些比特出错的位置是随机的, 则每个出错的位置 X_i 都是随机变量, 因此可以得到出错比特位置最大值与最小值之间距离是随机变量 X_i 的函数用 $f = X_{Max} - X_{Min}$ 表示, 其中, $X_{Max} = \text{Max}\{x_1, x_2, \dots, x_t\}$ 表示数据包中最大出错比特位置; $X_{Min} = \text{Min}\{x_1, x_2, \dots, x_t\}$ 表示数据包中最小出错比特位置。该函数值的期望可以用下面的公式表示:

$$E[f] = E[X_{Max} - X_{Min}] = E[X_{Max}] - E[X_{Min}] \quad (4.2)$$

由于我们设每个出错的比特位置 X_i 在数据包中是随机产生的, 因此可以认为随机变量 X_i 是独立同分布的, 每个位置发生错误的概率都相同, 该随机变量服从均匀分布, 表示为 $X \sim U[0, n]$, 其概率密度函数为 $p(X) = \frac{1}{n}$, 累积概率分布函数为 $F(X) = \frac{X}{n}$ 。由于各随机变量之间独立同分布, 故最小出错比特位置的累计分布函数可以表示为: $F_{Min}(X) = 1 - (1 - \frac{x}{n})^t$, 同理, 可以得到最大出错比特位置的累计分布函数 $F_{Max}(X) = (\frac{x}{n})^t$ 。最大出错比特位置的期望可以表示为:

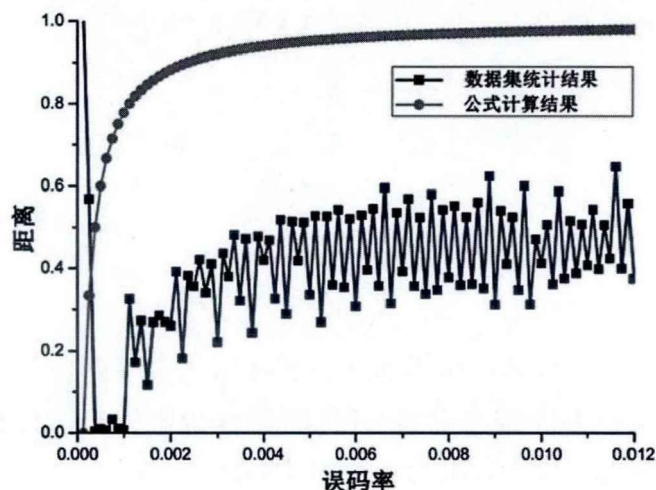


图 4.7: 出错数据比特之间的最大距离

$$\begin{aligned}
 E[X_{Max}] &= \int_0^n x F'_{Max}(X) dx \\
 &= \int_0^n x \frac{t}{n} \left(\frac{x}{n}\right)^{t-1} dx \\
 &= \frac{tn}{t+1} \left(\frac{x}{n}\right)^{t+1} \Big|_{x=0}^{x=n} \\
 &= \frac{tn}{t+1}
 \end{aligned} \tag{4.3}$$

最小出错比特位置的期望可以表示为:

$$\begin{aligned}
 E[X_{Min}] &= \int_{x=0}^{x=n} x F'_{Max}(X) dx \\
 &= \int_{x=0}^{x=n} x \frac{t}{n} \left(1 - \frac{x}{n}\right)^{t-1} dx \\
 &= nL \left[\frac{(1 - \frac{x}{n})^{t+1}}{t+1} - \frac{(1 - \frac{x}{n})^t}{t} \right] \Big|_{x=0}^{x=n} \\
 &= \frac{n}{t+1}
 \end{aligned} \tag{4.4}$$

所以, 根据公式4.3和公式4.4得到公式4.2的结果为:

$$\begin{aligned}
 E[f(X_t - X_1)] &= E[X_{Max} - X_{Min}] \\
 &= E[X_{Max}] - E[X_{Min}] \\
 &= \frac{(t-1)n}{t+1} \\
 &= \left(1 - \frac{2}{t+1}\right)n
 \end{aligned} \tag{4.5}$$

本文将 $E[f]$ 的统计结果和按照上述公式计算的结果做以对比得到图4.7, 在该图中, 红色的线表示在根据公式4.5 计算得到最小的出错位置和最大出错位置之间的距离的期望, 黑色线表示在真实WiFi访问日志中统计得到的真实距离。其中的纵坐标给出的距离的度量, 该值是错误比特距离与整个数据包长度的比值。横坐标表示在误码率。从图中可以看出, 如果假设数据包中的错误位置是随机的, 则与真实值之间有很大的差距, 因此可以得到的结论是: 在真实的WiFi数据中, 出错的比特之间的距离比随机错误之间的距离更近, 错误的位置并不是随机的, 错误与错误之间的距离更接近, 可以理解为发生了突发错误。

综上所述, 在真实的WiFi信道中, 如果有错误发生, 则错误比特之间距离很近, 即发生了突发错误, 由此可以推断, 在真实的WiFi信道是突发错误信道。

4.3 数据包修复协议的设计

在本节中, 我们设计了一种被称为EEC-PPR的基于估错码的出错包修复协议, 该协议与运行在链路层自动重传请求类似, 与当前广泛使用的802.11 中自动重传请求协议不同点如下:

- 使用估错码和分组校验对数据包编码;
- 在数据包中出现错误时, 当误码率足够小, 则不重传数据包, 当误码率较大则重传出错分组;
- 接收端可以向发送端发送错误确认报文 (NACK)。当数据包中出现错误时, 接收端将请求发送端重传错误所在的分组。

4.3.1 估错与分组

为了能够确定数据包中出错比特位的位置, 通过重传部分出错比特而不是重传完整的数据包, 就可以修复出错的数据包, 本文采用基于分组校验的方式。

根据4.2.2中的讨论, 我们知道如果采用分组的方式, 当误码率较小的时候, 通常只需要重传一个分组就可以纠正所有的错误; 当误码率大时, 通常只需要重传几个连续的分组就可以纠正所有的错误。

在每个分组中, 本文使用GS-EEC中的单层校验的方法判断该分组中是否包含有错误。即每个分组中设置若干个校验位校验该分组中的数据比特。使用这种方案是基于三方面的考虑:

- 冗余量尽可能的小: 由于每个分组中都需要添加冗余用以判断是否错误属于该分组, 因此如果每个分组中添加的冗余量大, 则整个数据包中的冗余将变得较大;
- 计算尽可能的简单: 由于接收端在解码时, 需要判断每个分组是否包含有错误, 为了能够减少系统的计算开销, 加快解码过程, 计算复杂度需要尽可能的降低;
- 适度的精确度: 对于错误位置判断而言, 越精准越好, 但是本协议所使用的场景是一些具有某种容错的应用, 因此即使出现了分组中存在错误但是没有被发现, 只要这种概率控制在一定的范围之内就是可容忍的。因此, 只要能够在较小的冗余下, 尽可能快的判断出出错的位置, 适当的假阳性错误是允许的。

下面将讨论对分组校验的具体方案。本文使用GS-EEC的单层校验方案, 而并不使用多层的方案。GS-EEC采用多层校验是为了能够更准确的估算误码率, 而在分组校验中, 只需要知道是否存在错误。因此, 用于捕获不同大小误码率的多层方案并不适用于分组校验。其次, 对于广泛使用的循环冗余校验而言, 循环冗余校验码能够判断的突发错误的长度与其除数多项式的阶数相关, 因此要提升循环冗余校验码的判断能力, 必须要增长除数多项式的阶数, 这样将增大计算量, 增大冗余度, 与设计目的不相符合。因此我们使用GS-EEC的单层校验方法。

我们通过下面的分析, 得到使用单层GS-EEC作为分组校验方案的性能。分析和实验显示, 该方案达到了预期的设计期望。假设每个分组大小为 S_{blk} , 每个分组共分配 t 位校验位, 每个校验位随机从分组中选择 s 比特校验。考虑对于每一位校验位, 记为 c_0 , 能够发现分组中出现错误的概率为

$$Pr(\text{find error}) = \sum_{i=1}^s \sum_{i \text{ is odd}} \binom{s}{i} p_b^i (1 - p_b)^{s-i} = \phi(s, p_b) \quad (4.6)$$

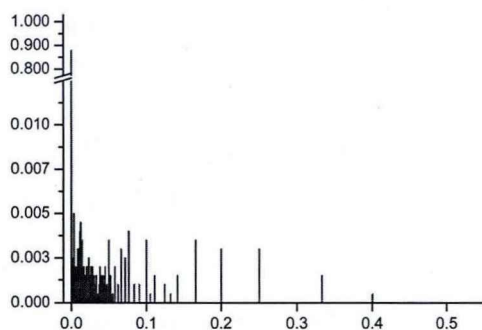


图 4.8: 校验分组失败率分布图

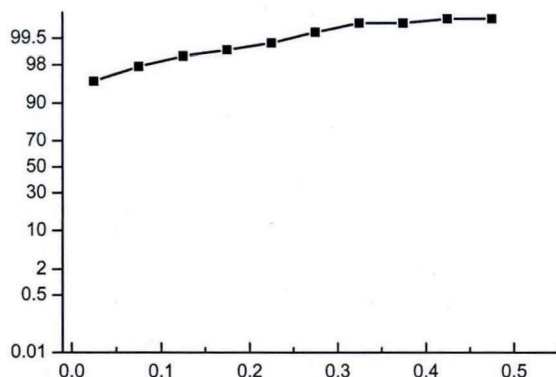


图 4.9: 校验分组失败率累积分布

其中 p_b 表示在该分组中, 误码率的大小, 一般而言, 由于错误多属于突发类型, 因此分组中的误码率一般大于整个数据包中的误码率。

- 当 $p_b = 0$ 时, 显然公式4.6为0;
- 当 $p_b \leq \frac{1}{2}$, 公式4.6中, ϕ 函数是以 s 为自变量的增函数;
- 当 $p_b \geq \frac{1}{2}$, 公式4.6中, ϕ 函数是以 s 为自变量的减函数。

根据 ϕ 单调性的特点可以得到, 如果希望 ϕ 函数在分组中的误码率很小(单一比特错误发生)或者误码率很大时(突发错误发生), 都能够尽可能大概率地发现错误, 则每一个校验位校验的数据位的数量应该取中间值, 即 $s = \frac{S_{blk}}{2}$ 。由于每个校验位是随机的从分组中选择数据位校验的, 因此, 即使分组中存在错误比特, 校验位也可能因为误判而通过校验, 这种错误称为假阳性错, 其概率为: $p_{fail} = 1 - \phi(\frac{S_{blk}}{2}, p_b) \geq \frac{1}{2}$ 。则 t 个校验位校验某个出错的分组都因为误判没有发现错误的概率为:

$$Pr(all\ fail) = p_{fail}^t = (1 - \phi(\frac{S_{blk}}{2}, p_b))^t \quad (4.7)$$

设 $S_{blk} = 128$, 则出错的分组的误码率最小值为 $p_b = \frac{1}{128}$, 当 $t = 7$ 时, 公式4.7的值将小于0.07。在本文的协议中, 我们设置 $t = 10$, 则校验位在判断分组中是否存在错误时, 假阳性的概率将小于0.02。由于错误的突发性, 出错分组的误码率往往远远大于 $\frac{1}{128}$, 因此假阳性的概率将远远小于0.02。图4.8和图4.9的参数设置如下: 选择每个分组中的校验位 $t = 10$, 每个检验位随机选择 $s = \frac{S_{blk}}{2}$ 大小作为校验参数时, 我们测试了分组校验方法的有效性。首先使用本文中提及的方法对每一个出错数据包进行编码, 这些数据包的误码率分布从 $\frac{1}{8000}$ 到 $\frac{1}{4}$, 其次我们对比真实的出错位置和分组校验方法给出的错误比特的位

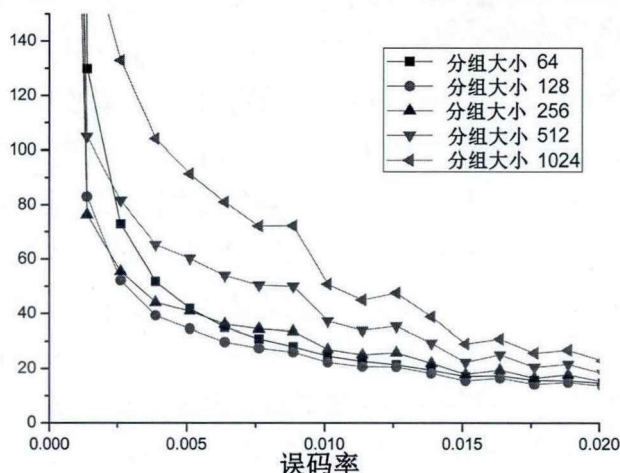


图 4.10: 分组不同大小下, 平均修复单个错误所需要的额外传输冗余

置, 得到分组校验假阳性的比例, 最后画出分布图和累计分布图。从图中我们可以看出, 分组校验能够正确判断出错误位置 (图中对应横坐标为0) 所占的概率接近90%, 假阳性率为其他值的比例很低。

关于分组大小, 很显然如果每个分组越大, 则出现错误需要重传时, 基于分组重传的方式将重传更多数据比特。如果分组越大, 一个数据包将分成更少的分组, 需要总的分组校验比特将减少。该结论反之亦然, 如果分组变小, 则出现错误时需要重传的数据变少, 但同时将导致所需的校验比特数目增加。同时考虑重传数据比特的大小和分组编码所需的校验冗余, 我们对分组大小的选取问题描述为最优化问题如下, 设每个分组的大小为 S_{blk} , 包含有错误的数据包数量为 N_{eblk} , 每个分组设 t 个校验位, 则每修复一个错误比特, 需要的额外冗余量可以表示为 $\frac{S_{blk}N_{eblk} + \frac{nt}{S_{blk}}}{np_0}$, 以分组大小作为参数, 求使得上述公式最小的参数的公式如下:

$$\operatorname{argmin}_{S_{blk} \in \{64, 128, 256, 512, 1024\}} \frac{S_{blk}N_{eblk} + \frac{nt}{S_{blk}}}{np_0} \quad (4.8)$$

按照公式4.8, 通过设定分组大小, 经过测试, 结果如图4.10所示。从图中可以很清晰的看出, 当分组大小为128时, 平均修复一个比特的错误, 所需要的传输冗余量最小。当分组大小为1024时, 所需要的传输冗余最大。随着误码率的增大, 各个分组大小导致的传输冗余量趋于相同。因此, 本文选取分组大小为128比特。

综上所述, 通过选取适当的参数, 使用分组单层估错码校验的方式, 就可以较为准确的判断出错误位置位于哪个分组中, 从而支持本文提出的分组重传的策略, 同时保证在重传中的信息冗余尽可能的小。

4.3.2 对估错算法的更新

本文的第三章中, 我们讨论了GS-EEC的纠错策略及其性能, 在本章的前一部分, 本文讨论为了能够确定错误的位置而设置的分组校验的方法。直观的看, 由于基于分组校验能够较为准确的判断出错的位置, 因此对纠错策略中过滤器算法将有较大的帮助。本小节将如何讨论利用分组校验信息对第三章中的算法做出必要的修改, 以充分的利用分组校验信息, 更准确的纠正错误数据比特。本小节中还将讨论修改后GS-EEC的纠错性能, 选取合适的阈值, 使得当误码率小于该阈值时, 则接收端将利用GS-EEC和分组的校验信息纠错, 当误码率大于该阈值时, 则接收端将不进行纠错, 而是通过分组校验的信息将出错的分组信息反馈给发送端, 由发送端重传相对应的分组, 完成对出错数据包的修复。

我们对第三章提出的过滤器进行更新, 增加一条新的规则如下:

规则3. 分组校验规则: 如果某个数据比特处在某个分组中, 并且该分组中的所有校验位都通过了校验, 则认为该数据位在传输过程中没有出错, 是安全的数据位。

设在接收端对数据包进行分组校验后, 得到的分组校验结果矩阵为

$$H = \begin{pmatrix} H_1 \\ H_2 \\ \dots \\ H_{\frac{n}{s_{blk}}} \end{pmatrix} = \begin{pmatrix} h_{1,1} & h_{1,2} & \dots & h_{1,t} \\ h_{2,1} & h_{2,2} & \dots & h_{2,t} \\ \dots & \dots & \dots & \dots \\ h_{\frac{n}{s_{blk}},1} & h_{\frac{n}{s_{blk}},2} & \dots & h_{\frac{n}{s_{blk}},t} \end{pmatrix} \quad (4.9)$$

其中每一个元素表示某个特定的分组校验位是否通过, 其取值与第三章中3.1校验结果矩阵中的项取值的意义相似, 所不同的是, 分组校验结果矩阵中的每一项表示该校验位对某个分组中的校验结果, 第*i*行的行向量 H_i 表示对分组 b_i 的分组校验结果。

$$h_{i,j} = \begin{cases} 0 & \text{如果分组 } b_i \text{ 的第 } j \text{ 个校验位通过校验} \\ 1 & \text{如果分组 } b_i \text{ 的第 } j \text{ 个校验位未通过校验} \end{cases} \quad (4.10)$$

规则3中, 如果分组 b_i 中的所有分组校验位都通过校验, 则 b_i 中的上所有数据比特 d 都是正确的。更形式化的描述为: $\forall d \in b_i$, 是正确比特当且仅当 $\sum_{j=1}^{j=t} h_{i,j} = 0$

由于增加了规则3, 因此过滤器算法的性能将会受到一定的影响。一个显然的结论是, 由于分组校验的存在和错误的突发特性, 将会使得过滤器算法的性

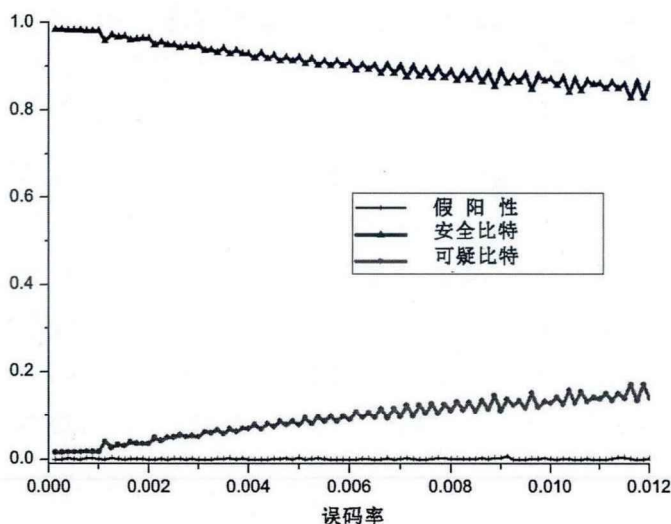


图 4.11: 添加分组校验规则后过滤器的性能

能变得更加优良。例如：当整个数据包中只有一个错误比特时，很显然规则3将排除 $n - S_{blk}$ 个数据比特，规则1和规则2仅能排除部分的正确数据。如果出错的数据比特为2个，则根据错误的分布情况，规则3最少将排除 $n - 2S_{blk}$ 个数据比特，如果错误是突发的，则规则3将排除 $n - S_{blk}$ 个数据比特。可以理解为，规则3将以很小的计算代价，极大的减少可疑数据比特的数量。图4.11中展示了规则3的过滤性能。从图中可以看出，假阳性的比例非常低根据实验的数据可以得到，假阳性的取值保持在往往小于 10^{-4} 数量级，这意味着，在随机翻转修复出错比特时，将更可能100%修复数据包中的错误数据，而不是只减少了错误的数量；经过规则3，可以看出，安全比特在误码率很小的情况下（例如误码率小于0.006），占了超过90%以上的，可疑数据比特所占比例仅不足10%，相比规则1与规则2的过滤性能(经过规则1和规则2后，可疑比特数量为25%左右)，规则3使得过滤器的性能有了很大的提高，这将极大的降低随机翻转过程中的时间复杂度。

第三章提出的估错码纠错方案分为两个步骤，首先使用过滤器，将收到的数据比特划分为两个集合：可疑比特集合和安全比特集合，分组校验的信息对过滤器性能有很大帮助。通过使用规则3，过滤器将对出错比特的搜索空间从整个数据包的大小减少到原有的10%左右。由于进一步降低了出错比特的搜索空间，运行在可疑比特集合上的随机翻转算法的速度将有所提高。同时，对比使用规则3前后的假阳性的比例，由于假阳性的比例进一步降低，因此随机翻转

算法能够更准确的翻转到出错的数据, 而不是将部分出错比特遗漏, 由此带来随机翻转算法准确性的提升; 再次基于分组的奇偶校验比特本身携带的冗余信息将在对错误位置的确认提供更多的信息, 综合上述分析可以得到, 在添加了分组校验码之后, 随机翻转算法的性能将有很大的提升。

为了能够使用分组校验码增强随机翻转的性能, 下面将第三章提出的3.1随机翻转算法做简单修改。算法3.1第4步中, 通过穷举的方法, 尝试可疑集合中的每一种可能的比特组合, 并翻转这些比特, 计算公式3.3中 $f(C, -1)$ 的值, 并以该函数作为评分函数, 比较随机翻转算法的每次的输出, 并选取最优结果作为最终的输出。分组校验的校验位也能够判定, 所选定的数据比特组合是否为真正的出错比特, 因此可以利用这些校验位设计新的评分函数:

$$g(H) = \sum_{i=1}^{\frac{n}{s_{blk}}} \sum_{j=1}^t h_{i,j} \quad (4.11)$$

在公式4.11的右半部分, $h_{i,j}$ 是分组校验结果矩阵中的项, 该项取值如果为“1”, 则表示存在一个校验位没有通过校验。随机翻转算法通过翻转可疑集合中的数据比特, 最小化公式4.11的值, 就能够使得出错数据包中的错误数量减少。为了最大限度兼容算法3.1, 本文结合公式3.3和公式4.11, 得到新的最小化目标函数如下:

$$\begin{aligned} F(C, H, -1) &= f(C, -1) + g(H) \\ &= \left| \sum_{\forall i \in [1, n], j \in [1, s \times l], c_{i,j} = -1} c_{i,j} \right| + \sum_{i=1}^{\frac{n}{s_{blk}}} \sum_{j=1}^t h_{i,j} \end{aligned} \quad (4.12)$$

本节中, 根据分组的校验位提供的信息, 在过滤器中增加规则3, 并修改了最小化目标函数, 通过使用过滤器算法和随机算法, 试图进一步减少出错数据包中的错误数量。我们使用第三章实验相同的设计, GS-EEC的参数设定略有不同: 每层的设定16比特的校验位, 共设定10层(标准的GS-EEC每层有32比特校验位, 共有 $\log n = 13$ 层的校验位)测试添加规则3, 更新最小化目标函数后, 本纠错策略的纠错性能。同样, 我们使用真实的WiFi访问日志作为测试样本。更新后的纠错策略对数据包的修复率如图4.12所示。在该图中当误码率小于0.012时, 数据包的修复率超过80%; 特别地, 当误码率 $p_0 \in (0, 0.003]$ 时, 数据包的修复率超过95%以上。与第三章中仅用GS-EEC的纠正策略相比, 性能有了较大的提升。我们认为, 在同时使用估错码和分组校验的情况下, 使用本文提出的过滤器算法和随机翻转算法在误码率极低的情况下, 具有较好的纠错性

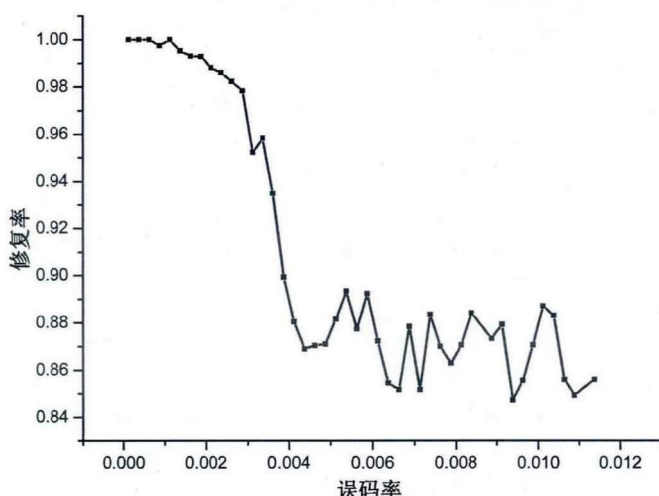


图 4.12: 使用分组校验后纠错策略的性能

能。在误码率较大的情况下，能够在一定程度上，减少数据包中的错误数量。在一些具有容错能力的应用场景中，可以将估错码的纠错能力与自动重传请求相结合，当收到的数据包的误码率很小时（如误码率小于0.003时），估错码一定的纠错能力可以以很大的概率纠正这些错误，如果收到的数据包中误码率较大，则可以选择重传某些分组从而达到数据包的修复。

这里可以看到，选用何种纠错策略取决于误码率的大小，本文的后续章节中，在讨论将估错码的估错、纠错能力与自动重传请求结合的具体内容时，将以0.003作为误码率大小的阈值，如果误码率大于0.003，则认为估错码可以修复数据包中的错误，如果误码率超过了0.003，则认为估错码不能完全修复数据包中的错误，为了节省计算资源，使用基于分块重传的方式修复出错的数据包，以此来修复出错的数据包，提高网络的性能。

4.3.3 基于估错码的自动重传请求

本小节将给出基于GS-EEC的自动重传请求协议的具体内容，包括发送端编码、发送以及接收端校验、修正数据包等。图4.13中给出了基于GS-EEC的自动重传请求的策略。其中，发送端在得到上层应用给出的数据包后，首先使用GS-EEC对数据包做估错码编码，被估错码编码过的数据包将再次被发送端分成若干分组，每个分组添加固定数量的校验位，随后经过两轮编码后的数据包将作为最终结果发送。在数据包发送后，发送端仍然将副本缓存在本地，同时发送端将维护一个计时器，等待来自接收端的确认信息。如果计时器超时，发送端将重新发送这个副本，连续三次超时将丢弃这个数据包，结束对该数据包

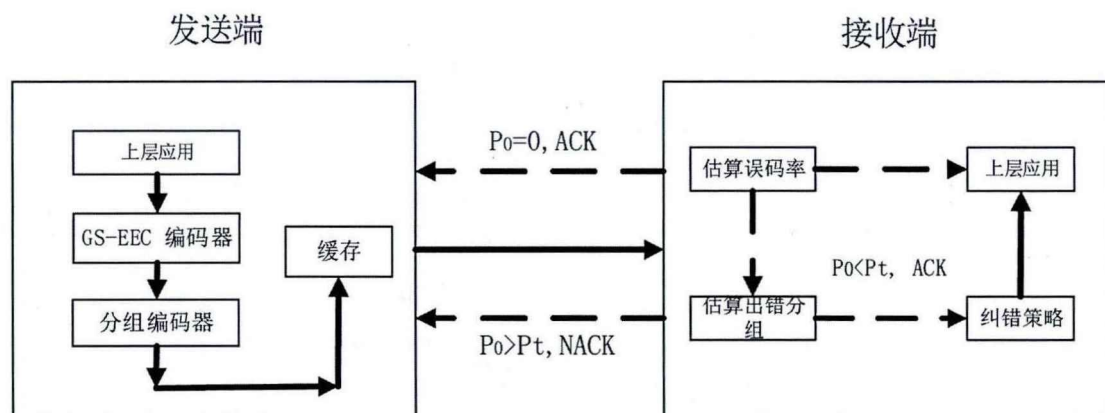


图 4.13: 基于GS-EEC的自动重传请求策略

的传输尝试。在该策略中，发送端必须能够处理两种确认信息即ACK和NACK。ACK用于接收端通知发送端正确接收当前数据包，和期望得到下一个数据包的编号；NACK用于接收端通知发送端当前数据包已经接收，但是数据包出错，期望得到当前数据包中给定部分的副本。当前的802.11中数据链路层的自动重传请求只能处理ACK而没有设计处理NACK。

在前文中，根据平均每传输一个数据比特需要的冗余最小作为衡量指标，确定每个分组大小为128比特，则每个数据包最多有100个分组，在NACK中使用比特图表示分组的接收情况，例如：当比特图中的第 i 位为“0”则表示正确接收，如果第 i 位为“1”则表示该分组中出现出错。因此，相比ACK，NACK中需要携带额外的13字节信息量，NACK长度为ACK长度的2倍。相对于数据帧而言，NACK仍然可以被视为短帧，NACK可以按照ACK类似的发送方法，在接收端收到出错的数据包时，经过计算获取出错分组的位置后，等待SIFS长度的时间就可以发送NACK。通过这种方法，实际上兼容了当前正在使用的802.11的重传请求协议：对于能够处理NACK的发送端，将重传NACK中所指定的分组，对于无法处理NACK的接收端，由于NACK的格式与ACK的格式不相同，因此将丢弃该确认，等待直到计时器超时，重传整个数据包。

对于接收端而言，如图4.13所示，当接收到数据包后，首先按照分组校验和GS-EEC的解码方法获得奇偶校验位和原始数据，估算出误码率 $\hat{p}_0$ 的大小，并根据误码率大小选择不同的操作。

- 当误码率为0时，证明数据包正确传输，则通过解码，原始数据将被交付到上层协议中，并向发送端发送ACK通知发送端正确收到了数据包，并期望收到下一个数据包的编号。

- 如果误码率不为0, 则根据GS-EEC估算的误码率 $\hat{p}_0$ 和第三章和4.3.2中的结论。
 - 当 $\hat{p}_0 \leq p_t$, 则说明数据包中出错的比特足够少, 使用基于GS-EEC的错误修复策略, 就可以在不需要重传的情况下, 以极大的概率修复数据包中的错误。因此, 数据包将使用纠错策略尽可能的纠正数据包中的错误, 并将修复的结果提交到上层应用, 同时向发送端发送ACK, 确认正确收到了数据包, 并期望收到下一个数据包。需要说明的是, 存在这样的情况, 即使经过修复策略, 数据包中也可能仍然有错误存在, 但是这样的概率较低, 且我们假设上层的应用具有一定的容错性能, 因此, 提交的数据包尽管可能包含有错误, 仍然可用。
 - 当 $\hat{p}_0 \geq p_t$, 则表示出错数据的数量超过了GS-EEC的纠错能力, 这种情况下使用纠错策略, 虽然能够降低数据包中出错比特的数量, 但是由于计算量较大, 占用了较多的计算资源, 因此接收端将采取重传的方式修复出错的数据包。接收端首先检查分组校验码, 并确定错误集中的分组的位置, 其次使用NACK向发送端通报出错的分组的位置, 并期望发送端重传出错的分组。

对于发送端而言, 如果收到了来自接收端的NACK, 并从该反馈中得知需要重传的分组的位置, 则发送端将重传了需要的分组后, 仍然缓存当前数据包, 并将计时器时间清零, 重新等待接收端的反馈信息。如果接收端经过若干轮的重传后, 最终正确接收了数据包, 则向发送端发送ACK确认。当发送端收到来自接收端的ACK信息, 则说明当前的数据包正确接收, 开始下一个数据包的传输。

4.4 性能评估

本章基于估错码的估错和纠错技术提出了一种新的出错数据包修复方案EEC-PPR, 该方案在无线网络中减少因数据包出错而重传的次数。在本节中, 我们将对比本文所提出的基于估错码的出错数据包修复策略EEC-PPR 和文献[3]中提到的基于纠错码重传技术ZipTx 以及文献[4]中提出的基于分块重传技术的数据包修复策略Maranello。通过对比说明了本文提出的基于估错码的数据包的修复策略所具有的优势。我们在真实的WiFi访问日志上, 模拟了不同的修复策略的过程。实验证明, 本文提出的重传技术能够降低传输时延, 同时给出的编码方案所需要的冗余更少。

文献[4]中提到的Maranello 方案是基于循环冗余校验的检测错误, 使用分组编码确定错误的位置, 并利用重传分组的方式修复出错的数据包; [3]中提出的ZipTx 方案同样使用循环冗余校验检测错误的发生, 但ZipTx 需要检测数据包的误码率。ZipTx通过在数据包中插入一些已知的比特, 在接收端检测这些已知比特的错误比例, 从而估算出数据包的误码率大小。本文则使用估错码和分组校验码检测错误位置和估算误码率。从自动重传请求策略上对比, Maranello 通过使用检错码发现错误所在的分组, 然后使用NACK向发送端发送重传该分组的请求。本文与Maranello比较类似, 不同之处在于放误码率小于某一阈值时, 本文将直接纠错, 而不请求重传, 只有当误码率超过了估错码的纠错范围, 才会请求重传所需分组。ZipTx为了能够节省重传的数据量, 基于对信道的统计的经验值, 将先重传少量的纠错码, 根据对信道的统计, 文献[3]认为可以在较大的概率上保证解码正确, 从而降低了冗余; 如果接收端解码失败则说明接收端再请求重传更多的纠错码。此时, 通过估算误码率, 发送端重传足量的纠错码确保纠错成功。

由于不同的策略使用了不同的编码和重传技术, 导致在传输发生错误时处理的机制和代价也不尽相同。对自动重传请求性能的评估可从两个方面考量:

- 在相同的WiFi信道条件下, 正确传输单个数据包需要的平均重传的次数;
- 在相同的WiFi信道条件下, 正确传输每个比特需要的平均冗余量。

对于出错数据包的修复协议而言, 正确传输单个数据包需要的平均重传的次数越少, 则意味着在一次传输过程中的传输时延越少, 网络的负载也会因此而降低, 并提高网络的吞吐量。本文首先对不同策略进行模拟测量平均每个数据包的重传次数。该模拟是在不同的WiFi访问日志中进行的。我们从数据集中选取了三种基于不同速率参数收集的访问日志, 分别为802.11b 协议下, 下载流量为5Mbps上传流量为500kbps的数据集、下载流量为10Mbps上传流量为500kbps、下载流量为10Mbps上传流量为1024kbps的访问日志。我们根据三种WiFi日志提供的信息, 模拟不同的WiFi场景, 针对ZipTx和Maranello以及本文提出的修复策略, 计算出这些策略每成功发送一个数据包需要的平均重传次数。需要说明的是, 发送的数据包可能出错, 因此接收端请求发送端重传数据包, 从而修复收到的出错包; 如果重传的数据包如果较大, 也可能由于信道干扰等原因再次出错。为了能够模拟真实场景, 我们使用访问日志中包含的信道信息。根据访问日志中数据包是否包含错误, 从而决定在模拟中该数据包是否出错; 由于访问日志中的数据包是连续存储的, 因此我们可以得到数据包在重

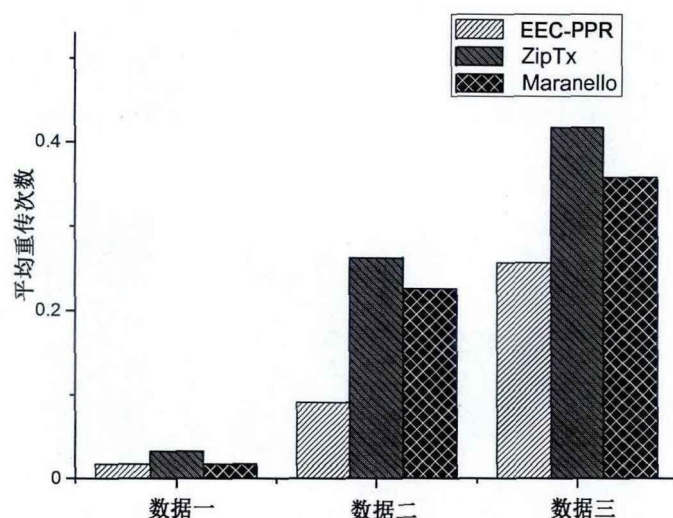


图 4.14: 不同数据包修复策略的平均重传次数

传时信道是否可靠,从而决定是否设定重传的数据包出错,是否需要重传。例如某个WiFi访问日志中第 i 个数据包出错需要重传,此时我们通过读取第 $i+1$ 个数据包并判断该数据包是否出错,就可以模拟第 i 个数据包在第一次重传时,是否也可能出错。通过对访问日志中包含的信道信息的模拟,可以使得本节中在对性能评估时,信道更加接近于真实情况。

图4.14中给出了本文提出的修复技术EEC-PPR和其他两种修复技术在平均重传次数方面的比较结果。对于不同的三种修复技术而言,平均的重传次数越少,则接收端每接收一个数据包需要的平均等待时间久越短,则数据传输也将越快,网络的吞吐量也将越高。结果显示,本文所提出的修复方案在不同WiFi访问日志中的重传次数最少。图中横坐标表示的三种不同的数据表示不同的访问日志。其中在数据一中,本文提出的EEC-PPR修复策略需要的重传次数不足0.02,但三种不同的技术重传次数较为接近,我们深入研究了数据一的特点,发现数据一出错的分布并不遵守幂律分布,前20%的误码率占据25%出错包,这种出错分布更类似于均一分布的特点。同时,数据一的出错数据包仅仅占整个数据集的不足2%。因此,三种修复技术在某种程度上趋于一致。数据二和数据三中,本文提出的基于EEC-PPR的重传策略与其他两种重传策略相比具有比较明显的优势,这说明使用本文提出的方案,系统的时延将更少,网络的吞吐量也将更大。

本节中提出的第二个性能评估指标为正确传输每个比特需要的冗余量。如果信道是完全可信的理想信道,则每个数据比特都可以以1比特的信息量传输到接收端。当信道中存在噪声或信道有干扰,为了能够正确传输每个比特,需要

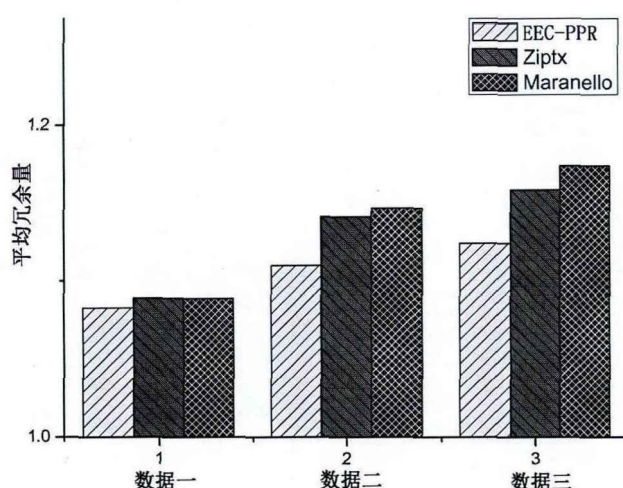


图 4.15: 不同出错包修复策略平均冗余量

在数据传输前添加冗余信息已确保数据在接收端能够正确解码, 这种思想常见于纠错编码中。在自动重传请求策略中, 在数据发送之前会添加检错码, 当接收端收到数据并发现其中包含有错误时, 则请求发送端重传该数据包。由于所有的额外信息都占用了网络的资源和带宽, 因此以上过程中为了保证接收端能够正确解码, 在原始的数据包中添加的冗余以及发送端重新发送的数据包的副本都被考虑在冗余中。

考虑到本文提出的出错包的修复技术中, 设置的估错码的参数为: 共使用10层, 每层用16比特估错码, 分组校验码共有64个, 每个使用7比特奇偶校验位。因此当数据包大小为8000比特时, 每个数据包因为编码索要添加的冗余为7.5%。重传的比特的数量在传输过程中动态的确定。对于Maranello中, 每个分组大小为64字节, 每个字节使用32位循环冗余校验作为检错码, 则整个过程中的编码冗余为6.9%。当出现错误时, Maranello只需要重传出错的分组, 修复冗余与重传出现错误的分组大小有关。在ZipTx的方案中, 分组和检错码使用相同的参数, 除此之外, ZipTx 为了能够较准确的获得误码率, 在数据包中增加了许多已知的数据比特, 通过这些比特的对错, 判断误码率的大小, 这部分冗余为0.8%, 因此该方案需要的编码冗余为7.8%。当发生错误时, ZipTx修复需要的纠错码冗余必须至少是错误数量的2倍。

图4.15中可以看出, 在三种不同的修复策略中, 本文提出的修复策略的平均的冗余量最少。平均冗余量是指, 在传输完一个数据的过程中, 相对于整个访问日志的数据的大小, 平均每正确传输一个数据比特, 需要传输多少个比特数据。从该图可以看出, 在数据一中, 由于信道非常可靠, 有98%的数据包能

够一次正确送达, 因此, 三种修复方案的冗余非常接近; 但是由于出错的数据包的错误比特非常集中, 因此本文提出的策略而言更有利。对于数据二和数据三出错的数据包分别为19%和28%, 由于出错的数据包数量变多, 三种修复策略需要修复的更多的出错比特, 因此相对于数据一而言, 冗余量有所增加。但是对于数据二和数据三, 由于误码率的幂律分布特性, 使得本文提出的策略能够不需要重传纠错码或者出错分组以修复出错比特, 而是直接使用估错码纠错, 由于错误的突发特性, 使得本文的策略能够只重传很少的分组, 就能够修复错误比特。对比基于分组重传的Maranello策略, 由于分组更大, 因此当出错率比较小的时候, 需要多重传更多的正确的数据比特, 而误码率较小的数据包占据非常大的比重, 这使得即使使用分组重传的方式, 也浪费了许多正确的比特, 从而冗余量更多。

4.5 本章小结

本章基于估错码的估错和纠错能力, 提出一种新的差错控制方案: EEC-PPR。通过对真实WiFi日志的分析, 发现WiFi信道中数据包误码率服从幂律分布, 即出错率前20%的出错包的数量占总的出错数据包的80%以上。同时通过理论分析与统计结果的对比, 得出数据包的错误属于突发错误类型, 即错误比特的位置不是独立的, 错误比特之间的距离非常接近, 甚至是连续发生的。本章将真实的WiFi信道的特点与估错码结合起来, 设计出EEC-PPR协议。

在该协议中, 首先使用估错码对原始数据编码, 其次对估错编码后的数据包分组校验。文章通过使用参数优化的方法设定分组大小, 使用单层的估错码对分组进行校验。在设计自动重传请求时, 引入NACK的短帧, 使得发送端能够在发现数据包中出现错误时, 通知发送端, 从而降低传输的时延, 提高网络的性能。接收端通过估错码判断数据包是否出错, 以及误码率的大小, 通过分组校验判断出错的比特位置。如果误码率小于给定的阈值, 接收端可以直接利用估错码的纠错算法修复错误, 从而避免重传带来的延时; 如果误码率大于给定的阈值, 接收端可以通过使用分组校验确定错误发生的分组, 并重传这些分组, 从而避免重传整个数据包, 有效减少重传的数据量。

本章使用基于真实的WiFi访问日志, 对三种不同的修复策略ZipTx, Maranello和本章提出的EEC-PPR做对比实验。本章中定义了两个评价指标: 每个数据包的平均重传次数和正确传输每个比特需要的冗余量, 实验表明, 在使用WiFi传输数据时, EEC-PPR所需要每个数据包的平均重传次数和正确传输每个比特需要的冗余量这两个指标方面都优于现有的工作, 即EEC-PPR重传技术能够减少重传次数, 降低传输时延; 减少整体的冗余量, 提高传输性能。

第五章 总结与展望

估错码是一种具有广泛应用前景的新型编码方案，该编码方案旨在估算数据包中的出错比特的数量——误码率。通过添加少量的冗余信息，经过复杂度较低的计算，接收端就可以计算出数据包中的误码率。由于误码率信息在许多应用场景中是非常重要的信息，因此估错码在许多场景中能够获得很好的收益。例如在基于误码率的WiFi速率调整算法中，通过对数据包的编码，接收端就可以计算并向发送端反馈误码率的大小，从而调整数据发送速率。在无线多跳网络中，基于误码率的数据包转发策略使得中间节点可以通过估算收到的数据包的误码率，决定是否转发数据包到下一节点，或是因为误码率太大，选择丢弃该数据包并且请求重传该数据包等等。

当前对误码率的研究集中在如何进一步降低估错码的编码冗余，降低编码和解码以及估算复杂度，提升估错码估算精度等几方面。研究人员并没有研究估错码所添加的冗余是否能够在某种程度上纠正出现的错误。估错码作为与纠错码权衡折中的结果，很自然的问题就是，估错码究竟有没有纠正错误的能力。本文回答了这个问题，在研究中我们发现，估错码具有一定的纠错能力，当误码率较小的时候，估错码能够以很高的精度纠正这些错误，当误码率较大时，估错码能够有效减少数据包中的错误数量，但是很难彻底纠正所有的错误。本文提出了一种基于估错码的纠错策略，我们首先将估错码的纠错问题转变为一种优化问题，其次将这一优化问题分解为两个步骤：使用两种规则组成的过滤器过滤大多数正确比特，使得出错的比特位置集中在少数比特位置中，这些剩下的比特组成的集合称之为可疑比特集合。其次，对可疑比特集合中的数据比特应用随机翻转算法，按照计算得到的概率值，随机排除大量的数据比特，穷举最终剩下的数据比特组合，并测试这些比特组合判断他们是否是降低了错误的数量。通过大量重复这一过程，可以得到近似最优的解，即找到了出错的比特位置。文章给出了对纠错策略中涉及的两条规则和随机翻转算法的性能极其复杂度的理论分析，给出了相应的理论结果。

经过理论分析，本文对给出的估错码应用到出错数据包的修复场景中，取得了较好的结果。在无线网络快速发展的今天，用户对无线网络的延时和带宽提出了越来越高的要求。无线局域网中差错控制仍然使用与有线网络的局域网中的差错控制类似的自动重传请求协议。该协议在无线网络存在许多问题，例如无线信道不可靠将造成数据在传输过程中更容易出现出错，同时出错的位置

由于突发因素而集中在某一区域。因此传统的方案中，一旦出现错误比特就重传整个数据包的做法效率不够高。深入的分析真实的WiFi访问日志，我们发现，数据包出错具有某些规律。

- 数据包中的错误数量作为随机变量，服从幂律分布；
- 数据包中的错误往往属于突发错误。

这意味着估错码较弱的纠错能力可能将在不需要重传的情况下，纠正大多数数据包的错误，因为这些数据包中仅有很少的错误。如果数据包中的错误数量较多，而由于错误的突发性，只需要传输连续的，很短的出错位置就能够纠正出错的比特，而不需要重传整个数据包。本文将估错码应用到无线网络的错误修复场景中，并在真实的WiFi访问日志做模拟，发现基于估错码的错误修复能够减少重传次数，较少总的修复冗余量，进而提升网络的性能。

不可否认，本文中提出的估错码的纠错策略和基于估错码的错误修复协议存在一些不足，需要在今后的工作中逐步改进和完善。

1. 在使用分组的形式对数据包进行编码后，没有在纠错策略中充分的应用分组校验比特提供的信息，目前我们仅向过滤器算法中添加了一条新的规则，同时更新了目标函数，但是没有因为分组校验信息而修改随机翻转算法中对比特删除概率计算。
2. 在随机翻转算法中，非常重要的一点是如何确定不同比特的删除概率。在文章中，我们假设每个估错码校验位的校验结果提供相同的信息，因此，每个校验位具有相同的权值。而事实上，由于估错码分层等特点，使得处在不同层的校验位能够提供的校验信息量并不相同，例如处在低层的校验位的权值应该更大，处在高层的校验位提供的信息较少，因此权重应该较低。但是这样的分析将增加理论分析的难度，本文暂无讨论，在未来的工作中，我们将根据校验位所在的层的位置因素，确定数据比特的删除概率。
3. 在本文中的第四章所提出的数据包修复协议，虽然做了细致的讨论，但是我们只在真实的WiFi访问日志中做了模拟，并没有实现真实系统。在未来的工作中，希望能够实现真实的原型系统，并在真实的环境中测试该协议的性能。
4. 在本文中缺乏对估错码纠错能力的理论证明。本文通过给出一种纠错策略证明估错码具有纠错能力，但是没有能通过理论证明估错码纠错能力的强

弱,在今后的工作中,希望能够使用通信复杂度或者信息论方面的知识,从理论上得出估错码的纠错能力的强弱。

总之,估错码具有非常广泛的使用前景,尽管在目前对估错码的估错能力等方面做了相关的研究,估错码的纠错能力缺乏充分的认识和研究,本文首先提出了估错码的纠错能力,并给出了一种估错码的纠错策略。通过实验发现估错码具有一定的纠错能力,我们将估错码的纠错策略应用到无线网络的差错控制这一经典问题上,设计出能够在不需要重传整个出错的数据包的情况下,修复出错的数据包,充分利用含有错误的数据包,从而在网络传输过程中减少了重传次数,降低了因为重传导致的时延,提高了网络的吞吐量。

附录 A WiFi访问日志收集的真实场景设置

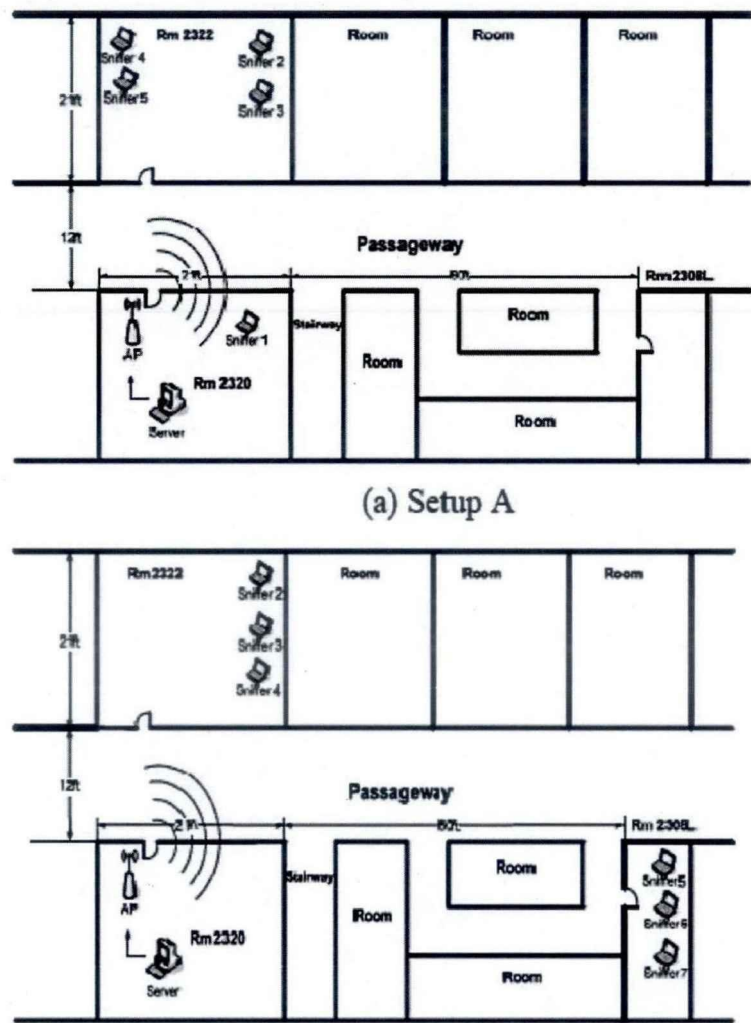


图 A.1: WiFi访问日志收集的真实场景设置[48]

参考文献

- [1] 艾瑞咨询集团. 中国智能手机市场研究报告简版. 2012.
- [2] Kyle Jamieson and Hari Balakrishnan. Ppr: Partial packet recovery for wireless networks. In *ACM SIGCOMM Computer Communication Review*, volume 37, pages 409–420. ACM, 2007.
- [3] Kate Ching-Ju Lin, Nate Kushman, and Dina Katabi. Ziptx: Harnessing partial packets in 802.11 networks. In *Proceedings of the 14th ACM international conference on Mobile computing and networking*, pages 351–362. ACM, 2008.
- [4] Bo Han, Aaron Schulman, Francesco Gringoli, Neil Spring, Bobby Bhattacharjee, Lorenzo Nava, Lusheng Ji, Seungjoon Lee, and Robert Miller. Maranello: practical partial packet recovery for 802.11. In *Proceedings of the 7th USENIX conference on Networked systems design and implementation*, pages 14–14. USENIX Association, 2010.
- [5] Binbin Chen, Ziling Zhou, Yuda Zhao, and Haifeng Yu. Efficient error estimating coding: feasibility and applications. In *ACM SIGCOMM Computer Communication Review*, volume 40, pages 3–14. ACM, 2010.
- [6] Hongyi Yao and Tracey Ho. Error estimating codes with constant overhead: A random walk approach. In *Communications (ICC), 2011 IEEE International Conference on*, pages 1–5. IEEE, 2011.
- [7] Nan Hua, Ashwin Lall, Baochun Li, and Jun Xu. A simpler and better design of error estimating coding. In *INFOCOM, 2012 Proceedings IEEE*, pages 235–243. IEEE, 2012.
- [8] Nan Hua, Ashwin Lall, Baochun Li, and Jun Xu. Towards optimal error-estimating codes through the lens of fisher information analysis. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE joint international conference on Measurement and Modeling of Computer Systems*, pages 125–136. ACM, 2012.

- [9] Zhenghao Zhang, Wei Hu, and Jin Xie. Employing coded relay in multi-hop wireless networks. *arXiv preprint arXiv:1012.4136*, 2010.
- [10] Jin Xie, Wei Hu, and Zhenghao Zhang. Revisiting partial packet recovery in 802.11 wireless lans. In *Proceedings of the 9th international conference on Mobile systems, applications, and services*, pages 281–292. ACM, 2011.
- [11] William Wesley Peterson and DT Brown. Cyclic codes for error detection. *Proceedings of the IRE*, 49(1):228–235, 1961.
- [12] Chung-Li Yu, Edward T Pak, and Ho-Ming Leung. Ecc/crc error detection and correction system, June 25 1991. US Patent 5,027,357.
- [13] Ross Williams. A painless guide to crc error detection algorithms. *Internet publication, August*, 1993.
- [14] Jay M Berger. A note on error detection codes for asymmetric channels. *Information and Control*, 4(1):68–73, 1961.
- [15] Richard W Hamming. Error detecting and error correcting codes. *Bell System technical journal*, 29(2):147–160, 1950.
- [16] Irving S Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the Society for Industrial & Applied Mathematics*, 8(2):300–304, 1960.
- [17] Marcel JE Golay. Notes on digital coding. *Proc. ire*, 37(6):657, 1949.
- [18] Tadao Kasami and Shu Lin. On the probability of undetected error for the maximum distance separable codes. *Communications, IEEE Transactions on*, 32(9):998–1006, 1984.
- [19] Robert Gallager. Low-density parity-check codes. *Information Theory, IRE Transactions on*, 8(1):21–28, 1962.
- [20] Yu Kou, Shu Lin, and Marc P. C. Fossorier. Low-density parity-check codes based on finite geometries: a rediscovery and new results. *Information Theory, IEEE Transactions on*, 47(7):2711–2736, 2001.
- [21] David JC MacKay and Radford M Neal. Near shannon limit performance of low density parity check codes. *Electronics letters*, 32(18):1645, 1996.

- [22] Shu Lin and Daniel J Costello. *Error control coding*, volume 123. Prentice-hall Englewood Cliffs, NJ, 2004.
- [23] Lalit Bahl, John Cocke, Frederick Jelinek, and Josef Raviv. Optimal decoding of linear codes for minimizing symbol error rate (corresp.). *Information Theory, IEEE Transactions on*, 20(2):284–287, 1974.
- [24] 庞宝茂. 现代移动通信. 清华大学出版社有限公司, 2004.
- [25] Peter Elias. Coding for two noisy channels. In *Information Theory, Third London Symposium*, pages 61–76. London, England, 1955.
- [26] John McReynolds Wozencraft. Sequential decoding for reliable communication. 1957.
- [27] James L Massey. Threshold decoding. Technical report, DTIC Document, 1963.
- [28] Andrew Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *Information Theory, IEEE Transactions on*, 13(2):260–269, 1967.
- [29] Claude Berrou and Alain Glavieux. Near optimum error correcting coding and decoding: Turbo-codes. *Communications, IEEE Transactions on*, 44(10):1261–1271, 1996.
- [30] 吴玉成, 杨士中, 刘嘉兴. 差错控制编码在卫星通信中的应用. 电讯技术, 40(1):3–7, 2000.
- [31] J Kurose and K Ross. *Computer Networks: A Top Down Approach Featuring the Internet*. Pearson Addison Wesley, 2006.
- [32] Amin Shokrollahi. Raptor codes. *Information Theory, IEEE Transactions on*, 52(6):2551–2567, 2006.
- [33] David Salomon. *Data compression: the complete reference*. Springer-Verlag New York Incorporated, 2004.
- [34] Noga Alon, Phillip B Gibbons, Yossi Matias, and Mario Szegedy. Tracking join and self-join sizes in limited storage. In *Proceedings of the eighteenth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, pages 10–20. ACM, 1999.

- [35] Grace R Woo, Pouya Kheradpour, Dawei Shen, and Dina Katabi. Beyond the bits: cooperative packet recovery using physical layer information. In *Proceedings of the 13th annual ACM international conference on Mobile computing and networking*, pages 147–158. ACM, 2007.
- [36] Allen Miu, Hari Balakrishnan, and Can Emre Koksul. Improving loss resilience with multi-radio diversity in wireless networks. In *Proceedings of the 11th annual international conference on Mobile computing and networking*, pages 16–30. ACM, 2005.
- [37] Shu Lin and P Yu. A hybrid arq scheme with parity retransmission for error control of satellite channels. *Communications, IEEE Transactions on*, 30(7):1701–1719, 1982.
- [38] Shyam S Chakraborty, Erkki Yli-Juuti, and Markku Liinajarja. An arq scheme with packet combining. *Communications Letters, IEEE*, 2(7):200–202, 1998.
- [39] Laurent R Lugand, Daniel J Costello Jr, and Robert H Deng. Parity retransmission hybrid arq using rate 1/2 convolutional codes on a nonstationary channel. *Communications, IEEE Transactions on*, 37(7):755–765, 1989.
- [40] David Chase. Code combining—a maximum-likelihood decoding approach for combining an arbitrary number of noisy packets. *Communications, IEEE Transactions on*, 33(5):385–393, 1985.
- [41] Sachin Katti, Dina Katabi, Hari Balakrishnan, and Muriel Medard. Symbol-level network coding for wireless mesh networks. *ACM SIGCOMM Computer Communication Review*, 38(4):401–412, 2008.
- [42] 林舒, 科斯特洛, 王育民, 王新梅. 差错控制编码: 基础和应用. 人民邮电出版社, 1986.
- [43] Brian P Crow, Indra Widjaja, LG Kim, and Prescott T Sakai. Ieee 802.11 wireless local area networks. *Communications Magazine, IEEE*, 35(9):116–126, 1997.
- [44] Henri Dubois-Ferrière, Deborah Estrin, and Martin Vetterli. Packet combining in sensor networks. In *Proceedings of the 3rd international conference on Embedded networked sensor systems*, pages 102–115. ACM, 2005.

- [45] Moncef Elaoud and Parameswaran Ramanathan. Adaptive use of error-correcting codes for real-time communication in wireless networks. In *INFOCOM'98. Seventeenth Annual Joint Conference of the IEEE Computer and Communications Societies. Proceedings. IEEE*, volume 2, pages 548–555. IEEE, 1998.
- [46] J Nicholas Laneman, David NC Tse, and Gregory W Wornell. Cooperative diversity in wireless networks: Efficient protocols and outage behavior. *Information Theory, IEEE Transactions on*, 50(12):3062–3080, 2004.
- [47] Rajeev Motwani and Prabhakar Raghavan. *Randomized algorithms*. Cambridge university press, 1995.
- [48] David Kotz and Tristan Henderson. Crawdad: A community resource for archiving wireless data at dartmouth. *Pervasive Computing, IEEE*, 4(4):12–14, 2005.

致 谢

首先我要由衷的感谢我的导师陆桑璐教授和李文中副教授和王晓亮博士。在我研究生的三年时光中，他们在学习上给予我悉心的指导，在工作中给予我充分帮助，在生活中给予我无微不至的关怀。我从对科研毫无了解到取得今天的进步，离不开他们的鼓励，帮助和指导。

感谢陆桑璐教授，她对待研究工作认真、务实，她严谨的治学态度、渊博的知识和勤奋的工作作风必将使我终生受益；同时陆老师对我们讲述的做人的道理，和我们分享的年轻人的成长轨迹和她的教育理念，我都将铭记于心，并将成为我未来宝贵的人生财富；特别要指出，陆老师为我未来的人生规划提供的各种良好的建议和引导，以及为此付出的努力，我将终身难忘。特别感谢陆老师能够在百忙之中抽出时间，阅读和修改我的论文，并提出大量修改意见。

其次，我要特别感谢李文中副教授，一位治学严谨、充满创新想法，知识渊博的学者，一位精力充沛、和蔼可亲，细致入微的老师。本文的研究工作是在他的悉心指导下完成的。从论文的选题、研究思路和方法、论文组织到资料查阅、论文结构和内容的修改直至最后的完善，他付出了大量的心血和无私的劳动，我衷心地感谢他对我的帮助和指导！也衷心感谢他对我的人生规划给予的良好建议和引导。

我还要特别的感谢王晓亮博士对我的大力帮助。他平易近人，研究思路清晰，治学严谨求实，见解独到。总是能指出我工作中的不足，针对本文的工作，王老师主动和我做了大量有益的讨论并指引我做出必要的修改，对我的工作最终定稿起了很大作用。由衷的感谢王老师给予我真诚的帮助和指导。

我要真诚的感谢叶保留教授从我本科毕业到读研期间对我生活的关怀和学习上的指导。他那废寝忘食的工作状态和忙碌的身影我一直无法忘怀，特别是在镇江研究院学习期间，他对我生活上关心和帮助使我难以忘怀。感谢顾庆教授在我做助教期间给予的帮助和信任，他的学者风范和渊博的知识使我钦佩。我还要感谢分布式计算实验室的钱柱中副教授、谢磊副教授对我的指导和帮助。特别感谢贾师傅在工作上对我的大力帮助。没有您的帮助，许多细碎的工作我无法完成。

感谢研究生学工办的四位老师对我工作的指导和生活关怀，他们是高海燕老师、曹迎春老师、尹存燕老师和乔益华老师。特别是高海燕老师给予我充分

的信任和最大的支持，鼓励我在工作中放手去做，对我工作的充分肯定，让我获得极大的自信心；在生活中，高老师给予我无微不至的关怀，为我提供许多勤工助学的岗位，帮助我解决物质生活的压力；在我的求职路上，高老师也多方关照，和我分享她的人生经验，并给予我许多中肯的建议，最终在她的帮助下我做出了正确的决定。感谢高老师三年时间对我的教诲和关心，这将是我一生的财富！

分布式计算实验室是一个团结而充满活力的大家庭。从我大四进入分布式计算实验室到研究生毕业共五年时间里，我的学长：李卓、游坤、郁奇峰、赵彦超、唐斌、徐天音、张胜、李鑫、王钦辉、胡跃飞、陈策、田宇、谭良良、牟权、孙正等，在工作、生活和学习上都给予我很大的帮助和指导，他们总是适时的和我分享学习和工作的心得，为我提供许多有益的建议，使我少走了许多弯路。感谢我的学长们，感谢来学长们的力量！还有我同级的十一位分布式的研究生，能和他们一起成长，共同经历研究生教育是我的荣幸。

感谢我充满活力的室友，能和你们一起生活三年是我最大的幸运。在人生仅有的研究生三年时光中，他们给予我无私的帮助和可靠的支持，使我回到宿舍，就像回到自己的家里；见到他们就像见到亲人那般亲切。从你们身上我看到了生活多姿多彩的一面，和他们许多有益的讨论促使我进步；和他们一起玩耍带来的欢声笑语是我一生美好的回忆。

最后，我要将最诚挚的感谢献给我的父母。多年来，他们对我的成长、工作、学习和生活付出了很多，没有他们含辛茹苦的培养和教导，我将一事无成。他们的教诲与关怀是我一生的财富，我希望他们能够真正为我感到骄傲与自豪。

仅以此文献给所有关心、帮助过我的人，以表达我由衷的谢意。

魏兴慎

2013年4月

